

Acoustic Scene Awareness: Urban and Household Sound Recognition using Convolutional Neural Networks with Autoencoder-Based Novel-Sound Detection

Softwarica College of IT & E-Commerce
(Coventry University)

Module: **STW7088CEM** – Artificial Neural Networks
Coursework Submission, M.Sc. Data Science and Computational Intelligence



Submitted by
SUDIP ADHIKARI
(250578)

Abstract

This report documents the design, implementation, and evaluation of a neural network system for recognising everyday urban and household sounds, motivated by assistive technology for Deaf and hard-of-hearing users who cannot rely on an audible cue when a siren, car horn, or alarm occurs. Raw audio is transformed into log-mel spectrograms — a two-dimensional representation of frequency against time — so that a convolutional neural network can learn each sound’s signature directly from the signal, without any hand-engineered acoustic features. Four tasks are performed on the UrbanSound8K and ESC-50 datasets: (i) ten-class **classification** with a four-block convolutional network; (ii) **novelty detection** using a convolutional autoencoder whose reconstruction error identifies sounds outside the training vocabulary; (iii) unsupervised **clustering** of the network’s learned embeddings using k-means and t-SNE; and (iv) an **architecture comparison** against a multi-layer perceptron baseline together with a data-augmentation ablation. All experiments follow UrbanSound8K’s official ten-fold cross-validation protocol, which is necessary because clips sliced from the same source recording share a fold; a random split leaks those slices across training and test and inflates reported accuracy. The convolutional network reaches 75.39% accuracy and 0.767 macro F1 while using 390,890 parameters — roughly fifteen times fewer than the 5,804,810-parameter perceptron baseline it outperforms, which quantifies the value of convolutional inductive bias on spectrogram input. The autoencoder detects previously unheard sounds with a ROC-AUC of 0.760, and is compared against the obvious alternative of thresholding classifier confidence. All models were trained locally on an Apple M1 laptop using the Metal Performance Shaders backend; no paid computing resources were used.

Contents

1	Introduction	3
1.1	Background and Motivation	3
1.2	Why This Is a Neural Network Problem	3
1.3	Project Objectives	3
1.4	Original Contributions	4
1.5	Report Structure	4
2	Background and Related Work	4
2.1	Environmental Sound Classification	4
2.2	Spectrograms as Network Input	4
2.3	Data Augmentation	5
2.4	Novelty and Out-of-Distribution Detection	5
2.5	Positioning of This Work	5
3	Datasets	5
3.1	UrbanSound8K	5
3.2	ESC-50 (Unseen Sounds)	6
3.3	Evaluation Protocol and a Common Methodological Error	6
4	Methodology	7
4.1	Audio Front-End	7
4.2	Data Augmentation	7
4.3	Architecture 1: Multi-Layer Perceptron (Baseline)	8
4.4	Architecture 2: Convolutional Neural Network	8
4.5	Architecture 3: Convolutional Autoencoder	8

5	Implementation	9
5.1	Environment	9
5.2	Project Layout	9
5.3	Feature Extraction	9
5.4	The Fold Split	10
5.5	The Convolutional Classifier	10
5.6	Novelty Scoring	11
5.7	Reproducibility	11
5.8	Project Resources	11
6	Results and Discussion	12
6.1	Architecture Comparison	12
6.2	Per-Class Performance and Error Structure	14
6.3	Clustering the Learned Representation	15
6.4	Novel-Sound Detection	16
6.5	Real-Time Demonstration	18
7	Limitations and Future Work	18
8	Social, Ethical, Legal, and Professional Considerations	19
8.1	Privacy	19
8.2	Asymmetric Consequences of Error	19
8.3	Bias and Dataset Provenance	19
8.4	Accessibility and Appropriate Framing	19
8.5	Licensing and Attribution	20
9	Conclusion	20
	References	20
A	Project Proposal (As Submitted)	22
B	Reproduction Instructions	22
C	Experiment Evidence	22
D	Source Code	22
D.1	src/config.py	22
D.2	src/data.py	24
D.3	src/models.py	28
D.4	src/train.py	31
D.5	src/evaluate.py	36
D.6	src/anomaly.py	37
D.7	src/cluster.py	40
D.8	src/viz.py	42
D.9	run_experiments.py	48
D.10	demo/live_demo.py	53

1 Introduction

1.1 Background and Motivation

Sound carries a large amount of safety-critical information that is normally processed without conscious effort. A siren approaching from behind, a car horn, a smoke alarm, glass breaking in another room, or somebody knocking at the door are all events that a hearing person reacts to automatically. For a person who is Deaf or hard of hearing, none of these signals arrive. The World Health Organization estimates that over 1.5 billion people live with some degree of hearing loss, and the everyday consequence is not merely inconvenience but a genuine reduction in situational safety.

Commercial systems already exist in this space — Apple’s Sound Recognition and Google’s Sound Notifications both listen for a small set of household sounds and raise a visual or haptic alert — which establishes that the problem is real and the approach is viable. What those systems do internally is not published, and building an equivalent from open data is a well-scoped exercise in applied neural network design.

1.2 Why This Is a Neural Network Problem

It is worth being explicit about why this task requires neural networks rather than the classical machine learning methods studied in other modules. Raw audio is a waveform sampled at tens of thousands of values per second; there is no meaningful way to present that directly to a decision tree, a support vector machine, or a logistic regression. Classical approaches to audio therefore depend on a human first deciding which summary statistics matter — zero-crossing rate, spectral centroid, a handful of MFCC coefficients — and the quality of the result is bounded by the quality of that hand-engineering.

A convolutional network removes that bottleneck. Presented with a spectrogram, it learns its own filters: early layers respond to local time–frequency texture such as onsets, harmonic stacks, and broadband noise, and deeper layers compose those primitives into class-level patterns. The representation is discovered from data rather than specified in advance. This property — *representation learning* — is the defining capability of artificial neural networks and the reason the field displaced feature-engineering pipelines in audio, vision, and language.

The novelty-detection task reinforces the point. An autoencoder learns a compressed internal code for the sounds it has seen and rebuilds the input from that code. Nothing in classical supervised learning provides an equivalent mechanism for saying “*this input is unlike anything I was trained on*” without being given negative examples in advance.

1.3 Project Objectives

1. Build a convolutional neural network that classifies ten urban sound classes from log-mel spectrograms, evaluated under the dataset’s official ten-fold protocol.
2. Quantify the benefit of convolutional structure by comparing against a multi-layer perceptron baseline of substantially larger parameter count.
3. Measure the effect of spectrogram-domain data augmentation (SpecAugment and time shifting) as a controlled ablation.
4. Implement novelty detection with a convolutional autoencoder so that unfamiliar sounds are reported as unknown rather than being forced into one of the ten trained labels, and compare this against a softmax-confidence baseline.
5. Examine the learned representation without labels, using k-means clustering and t-SNE projection of the network’s penultimate embeddings.

6. Deliver a working real-time demonstration that runs from a laptop microphone.

1.4 Original Contributions

The individual components — CNNs on mel-spectrograms, autoencoder anomaly scoring — are established techniques. The contributions of this work are in their combination and in the rigour of the evaluation:

- **An open-set framing of a closed-set benchmark.** UrbanSound8K is almost always treated as a ten-way classification problem. Here the classifier is paired with a novelty detector and evaluated against genuinely unseen sound classes drawn from a second dataset (ESC-50), which is the situation any deployed assistive device actually faces.
- **An honest baseline for the novelty detector.** Rather than presenting autoencoder reconstruction error in isolation, it is compared directly against thresholding the classifier’s own softmax confidence — the cheap alternative that a reviewer would immediately propose.
- **A parameter-matched argument for convolution.** The MLP baseline is deliberately larger than the CNN, so the comparison isolates architectural inductive bias rather than model capacity.
- **A real-time deployment.** The trained networks run live on laptop hardware with both models in the loop, demonstrating that the system is practical rather than purely offline.

1.5 Report Structure

Section 2 reviews related work. Section 3 describes both datasets and the evaluation protocol. Section 4 sets out the audio front-end and the three network architectures. Section 5 covers implementation with reference to the source code. Section 6 presents and discusses results across all four tasks. Sections 7 and 8 address limitations and the ethical dimension of always-on microphones, and Section 9 concludes.

2 Background and Related Work

2.1 Environmental Sound Classification

Environmental sound classification (ESC) differs from speech and music recognition in that the sounds have no shared grammatical or harmonic structure. Early systems used hand-designed features — most commonly Mel-Frequency Cepstral Coefficients (MFCCs), inherited from speech processing — fed into a Gaussian Mixture Model or Support Vector Machine.

The shift to neural approaches was driven by two datasets released in the mid-2010s. Salamon, Jacoby and Bello introduced UrbanSound8K [1], which remains the standard benchmark for urban ESC, and Piczak released ESC-50 [2] alongside a demonstration that convolutional networks operating on spectrograms outperform the feature-engineering pipelines that preceded them [3].

2.2 Spectrograms as Network Input

The dominant design pattern in modern ESC is to convert audio into a time-frequency representation and treat the result as an image. The mel scale is used because it approximates the non-linear frequency resolution of human hearing, allocating more resolution to lower frequencies where most environmental sound energy lies. Amplitudes are converted to decibels

because loudness perception is logarithmic; without this, a small number of high-energy frames dominate the input range and training becomes unstable.

Comparative studies of time–frequency representations for ESC [8] found log-mel spectrograms to be a consistently strong choice, which motivates their use here.

2.3 Data Augmentation

ESC datasets are small by deep learning standards — UrbanSound8K contains 8,732 clips, orders of magnitude fewer than the image corpora on which deep CNNs were developed — so overfitting is the central practical difficulty. Salamon and Bello [4] showed that audio-domain augmentation (time stretching, pitch shifting, adding background noise) materially improves CNN performance on UrbanSound8K.

Park et al. later introduced SpecAugment [5], which operates directly on the spectrogram by masking random frequency bands and time spans. It is considerably cheaper than audio-domain augmentation, since no re-synthesis or re-transformation is required, and it forces the network to use the entire time–frequency pattern rather than latching onto a single narrow band. SpecAugment and random time shifting are the augmentations used in this project.

2.4 Novelty and Out-of-Distribution Detection

A softmax classifier is closed-set by construction: its outputs are constrained to sum to one over the classes it knows, so an unfamiliar input is necessarily assigned to one of them. Hendrycks and Gimpel [6] established the baseline that maximum softmax probability carries *some* out-of-distribution signal, while also documenting that neural networks are frequently confident when wrong.

Autoencoder reconstruction error is a widely used alternative [7]: a model trained to compress and rebuild in-distribution data will reconstruct out-of-distribution data poorly, and the error is a usable score requiring no negative examples at training time. This report implements the autoencoder approach and evaluates it against the Hendrycks and Gimpel softmax baseline rather than assuming its superiority.

2.5 Positioning of This Work

Relative to the literature, this project sits at the intersection of standard ESC classification and open-set recognition. Reported CNN accuracies on UrbanSound8K commonly fall in the 70–92% range depending on architecture, augmentation, and — importantly — whether the official fold protocol was respected. This work targets the upper part of that range with a deliberately compact architecture, and extends the standard setup with the novelty-detection and clustering tasks required by the assignment brief.

3 Datasets

3.1 UrbanSound8K

UrbanSound8K [1] is the primary dataset: 8,732 labelled audio excerpts of up to four seconds, drawn from field recordings on the Freesound archive, covering ten classes selected from an urban sound taxonomy. The classes are `air_conditioner`, `car_horn`, `children_playing`, `dog_bark`, `drilling`, `engine_idling`, `gun_shot`, `jackhammer`, `siren`, and `street_music`.

Four of these — `car_horn`, `dog_bark`, `gun_shot`, and `siren` — are treated as safety-critical in the demonstration system and trigger a distinct alert state.

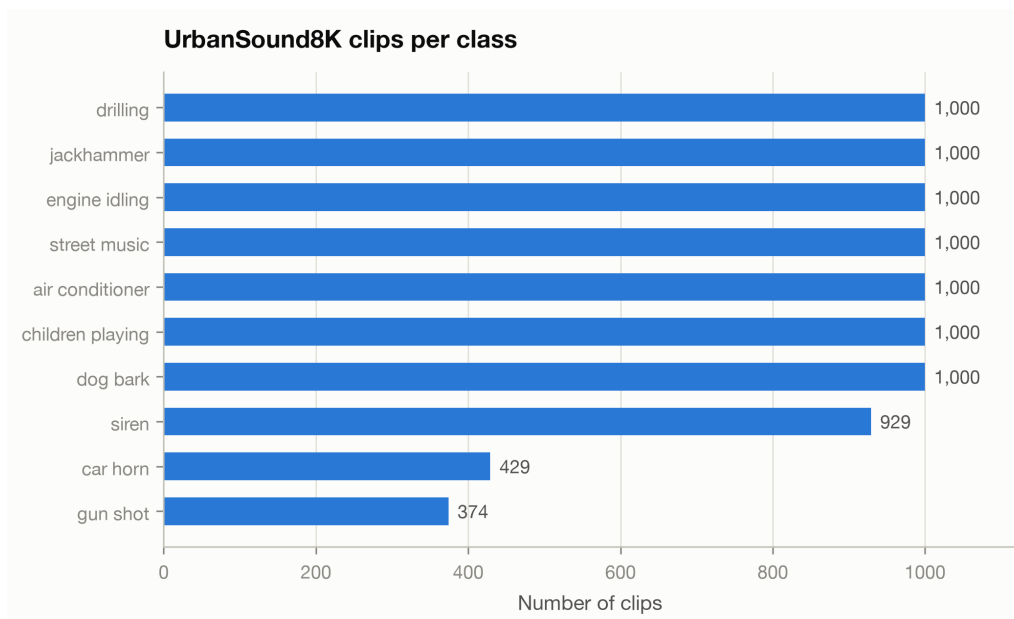


Figure 1: Clip count per class in UrbanSound8K. The set is broadly balanced apart from `car_horn` and `gun_shot`, which are noticeably smaller — a fact that becomes relevant when interpreting per-class F1 in Section 6.

3.2 ESC-50 (Unseen Sounds)

ESC-50 [2] contributes 2,000 five-second clips across 50 classes. It is *not* used for classifier training. Its role is to supply genuinely unfamiliar sounds for the novelty-detection experiment: eight household and safety-relevant classes were selected — `door_wood_knock`, `glass_breaking`, `crying_baby`, `clock_alarm`, `door_wood_creaks`, `water_drops`, `footsteps`, and `can_opening` — none of which appear in the UrbanSound8K vocabulary.

This choice is deliberate. Evaluating novelty detection against random noise would be trivially easy and uninformative; evaluating it against real recordings of plausible household events is a fair test of whether the system would behave sensibly in the environment it is intended for.

3.3 Evaluation Protocol and a Common Methodological Error

UrbanSound8K ships with ten predefined folds, and the dataset authors are explicit that these must be used rather than a random split. The reason is that the 8,732 clips are sliced from a much smaller number of source field recordings, and all slices from one recording are placed in the same fold. Under a random split, several slices of a single recording land in training while others land in test; the model can then succeed by recognising the specific recording rather than the sound class, and reported accuracy is inflated by a wide margin.

All results in this report use ten-fold cross-validation over the official folds. For each fold k , fold k is the test set, fold $(k \bmod 10) + 1$ is the validation set used for early stopping and model selection, and the remaining eight folds are used for training. Normalisation statistics are computed on the training partition alone and then applied unchanged to validation and test, so that no test statistics leak into training.

Because every clip appears in exactly one test fold, pooling the held-out predictions across all ten folds yields one honest prediction per clip for the entire dataset. The aggregate confusion matrix and per-class scores in Section 6 are computed on that pooled set.

4 Methodology

4.1 Audio Front-End

Every clip passes through an identical front-end before reaching any network:

1. **Load as mono at 22,050 Hz.** Halving the usual 44.1 kHz rate discards content above 11 kHz, which carries little discriminative information for these classes, and halves the computational cost.
2. **Fix the duration to four seconds.** Shorter clips are zero-padded and longer ones truncated, so every spectrogram has identical dimensions — a requirement for batching through a convolutional network with a fixed input size.
3. **Short-Time Fourier Transform** with a 1,024-sample window and 512-sample hop.
4. **Project onto 64 mel bands** between 20 Hz and the Nyquist frequency.
5. **Convert power to decibels**, referenced to the per-clip maximum.

The result is a 64×173 matrix per clip: 64 frequency bands by 173 time frames. Figure 2 shows one example per class, and makes the central argument of the project visually: each sound class produces a visibly distinct pattern, which is precisely what makes a convolutional network appropriate.

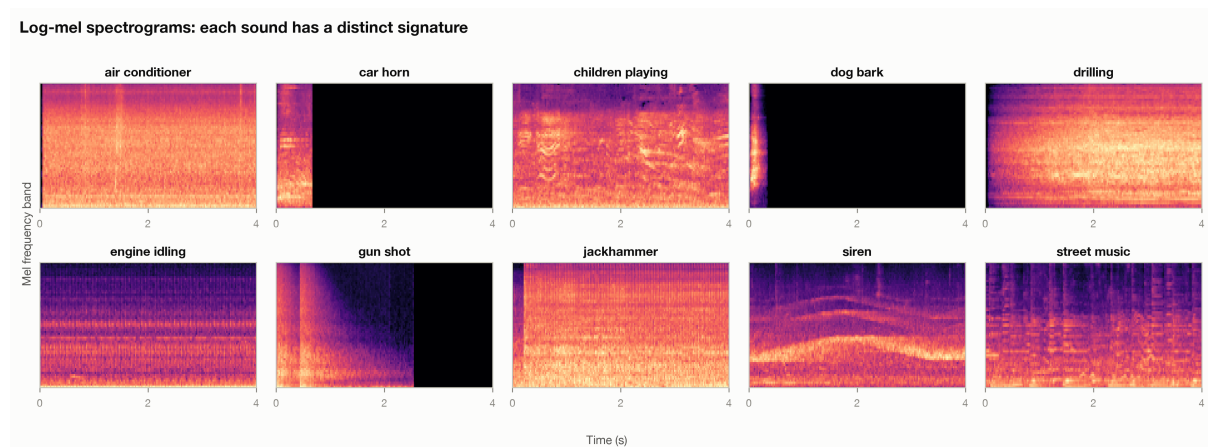


Figure 2: Log-mel spectrograms, one per UrbanSound8K class. The horizontal striations of `jackhammer` and `drilling`, the rising harmonic sweep of `siren`, and the sharp broadband transient of `gun_shot` are distinguishable by eye — structure that a convolutional network can exploit and that a flattened feature vector destroys.

4.2 Data Augmentation

Two spectrogram-domain augmentations are applied to training data only:

- **Random time shift** (up to $\pm 20\%$ of the clip), implemented as a circular roll. A siren remains a siren regardless of when within the window it begins, so the network should not depend on absolute position.
- **SpecAugment** [5]: one random frequency band (up to 8 of 64 mel bins) and one random time span (up to 16 of 173 frames) are masked to the spectrogram floor. This prevents the network from relying on any single narrow band and acts as a strong regulariser on a dataset of this size.

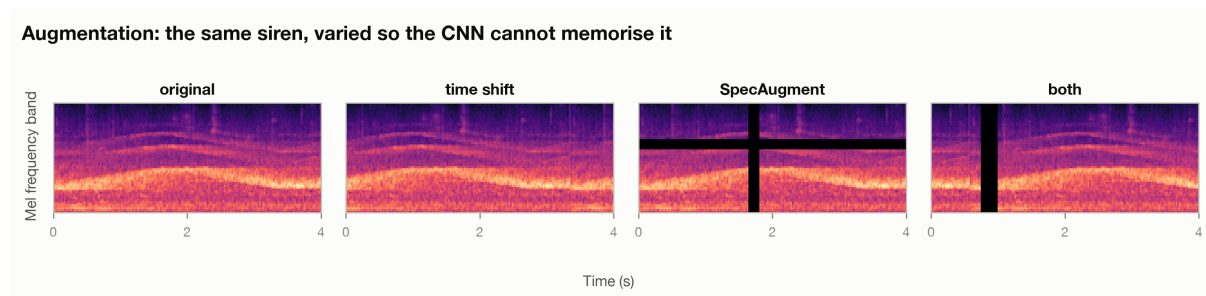


Figure 3: One siren clip under each augmentation. Masked bands and spans are visible as flat regions. The network never sees exactly the same input twice, which is what suppresses memorisation.

4.3 Architecture 1: Multi-Layer Perceptron (Baseline)

The baseline flattens the 64×173 spectrogram into an 11,072-dimensional vector and passes it through two fully connected hidden layers (512 and 256 units) with batch normalisation, ReLU activation, and dropout at $p = 0.5$.

Its purpose is diagnostic. Flattening destroys the two-dimensional structure of the spectrogram: the network has no notion that two adjacent time frames are related, or that a harmonic pattern shifted slightly in frequency is the same pattern. Every input pixel receives its own independent weight, which is why the model requires 5,804,810 parameters — roughly fifteen times the CNN’s 390,890. If the CNN outperforms it despite that disparity, the difference is attributable to architecture rather than capacity.

4.4 Architecture 2: Convolutional Neural Network

The main classifier consists of four convolutional blocks, each comprising a 3×3 convolution, batch normalisation, ReLU, and 2×2 max pooling, with channel widths of 32, 64, 128, and 256. Spatial dropout is applied in the deeper blocks.

Two design decisions are worth stating explicitly:

- **Global average pooling replaces a flatten-and-dense head.** Collapsing each of the 256 final feature maps to its mean gives a 256-dimensional descriptor. This keeps the parameter count an order of magnitude below the MLP and, more importantly, makes the model approximately invariant to *where* in the clip a sound occurs — appropriate, because a dog bark two seconds in is the same event as a dog bark at the start.
- **The 256-dimensional pooled vector is exposed as an embedding.** This is the representation analysed in the clustering task (Section 6.3), allowing the learned feature space to be examined directly.

Training uses AdamW (learning rate 10^{-3} , weight decay 10^{-4}), cross-entropy loss with label smoothing of 0.05, cosine-annealed learning rate, and early stopping on validation accuracy with a patience of eight epochs.

4.5 Architecture 3: Convolutional Autoencoder

The novelty detector is a symmetric convolutional autoencoder. The encoder applies three stride-2 convolutions (16, 32, and 32 channels), compressing the 64×173 input to a $32 \times 8 \times 22$ latent representation; the decoder mirrors this with transposed convolutions. Because transposed convolution can overshoot the original dimensions by a pixel, the output is cropped back to the input size.

Crucially, it is trained *only* on the ten known classes, with a mean-squared-error reconstruction objective. Having learned a compressed code adequate for those sounds, it reconstructs an unfamiliar sound poorly, and the per-clip reconstruction error

$$s(x) = \frac{1}{HW} \sum_{i,j} (\hat{x}_{ij} - x_{ij})^2$$

serves directly as a novelty score. The decision threshold is set at the 95th percentile of the reconstruction errors observed on *known* data — a calibration that requires no novel examples at all, matching the constraint a deployed system operates under.

At 28,321 parameters the autoencoder is by far the smallest of the three networks, which matters for running both models concurrently in the real-time demonstration.

5 Implementation

5.1 Environment

All work was carried out on an Apple MacBook (M1, 16 GB) running Python 3.11 and PyTorch 2.13, using the Metal Performance Shaders (MPS) backend for GPU acceleration on Apple Silicon. No cloud or paid computing resources were used at any stage.

One environment issue is worth recording for reproducibility: the Homebrew-supplied Python builds on the target machine had a broken `pyexpat` module, which links against the older system `libexpat` and fails to resolve a required symbol. This prevents `pip` from bootstrapping inside a virtual environment. The resolution was to use `uv` to provision a self-contained CPython 3.11 distribution, which is documented in the repository README.

5.2 Project Layout

<code>src/config.py</code>	hyperparameters, paths, device selection
<code>src/data.py</code>	audio loading, mel-spectrograms, augmentation,
<code>src/models.py</code>	MLPBaseline, AudioCNN, ConvAutoencoder
<code>src/train.py</code>	training loop and 10-fold cross-validation
<code>src/evaluate.py</code>	pooled metrics, confusion matrix, embeddings
<code>src/anomaly.py</code>	autoencoder novelty detection + softmax baseline
<code>src/cluster.py</code>	k-means, t-SNE, cluster quality metrics
<code>src/viz.py</code>	all report figures
<code>run_experiments.py</code>	staged experiment runner
<code>demo/live_demo.py</code>	real-time microphone dashboard

5.3 Feature Extraction

The front-end of Section 4 is implemented in `src/data.py`. Listing 1 shows the core transform.

```

1 def audio_to_melspec(y: np.ndarray, sr: int = C.SAMPLE_RATE) -> np.ndarray:
2     mel = librosa.feature.melspectrogram(
3         y=y, sr=sr,
4         n_fft=C.N_FFT,          # 1024
5         hop_length=C.HOP_LENGTH, # 512
6         n_mels=C.N_MELS,        # 64
7         fmin=C.F_MIN, fmax=C.F_MAX,
8     )
9     # Decibel scaling: loudness perception is logarithmic, and without
10    # this a few loud frames dominate the input range.
11    mel_db = librosa.power_to_db(mel, ref=np.max)
12    return mel_db.astype(np.float32)

```

Listing 1: Waveform to log-mel spectrogram (`src/data.py`).

Extraction runs once over the full corpus and is cached to `.npy`, after which every experiment reads features from memory. Measured throughput was approximately 3 ms per clip, so caching all 8,732 spectrograms completes in well under a minute — a useful property when experiments must be repeated.

5.4 The Fold Split

Listing 2 implements the protocol described in Section 3.3. The comment records *why* the split is by fold, since this is the single easiest way to produce misleadingly high numbers on this dataset.

```

1 def make_fold_split(X, meta, test_fold: int, val_fold: int | None = None):
2     """Split by UrbanSound8K's official folds.
3
4     Clips sliced from the same source recording share a fold, so splitting
5     by fold (rather than randomly) is what keeps the evaluation honest.
6     """
7     if val_fold is None:
8         val_fold = test_fold % C.N_FOLDS + 1
9
10    folds = meta["fold"].values
11    labels = meta["classID"].values
12
13    te = folds == test_fold
14    va = folds == val_fold
15    tr = ~(te | va)
16
17    return (X[tr], labels[tr]), (X[va], labels[va]), (X[te], labels[te])

```

Listing 2: Official fold-based splitting (`src/data.py`).

5.5 The Convolutional Classifier

```

1 class AudioCNN(nn.Module):
2     def __init__(self, n_classes: int = 10, dropout: float = 0.3, width: int =
3         32):
4         super().__init__()
5         w = width
6         self.block1 = ConvBlock(1, w, pool=2)
7         self.block2 = ConvBlock(w, w * 2, pool=2, dropout=dropout / 2)
8         self.block3 = ConvBlock(w * 2, w * 4, pool=2, dropout=dropout / 2)
9         self.block4 = ConvBlock(w * 4, w * 8, pool=2, dropout=dropout)
10
11        # Global average pooling instead of flatten+dense: ~10x fewer
12        # parameters, and invariance to where in the clip the sound occurs.
13        self.gap = nn.AdaptiveAvgPool2d(1)
14        self.dropout = nn.Dropout(dropout)
15        self.fc = nn.Linear(w * 8, n_classes)
16
17        def forward(self, x, return_embedding: bool = False):
18            x = self.block4(self.block3(self.block2(self.block1(x))))
19            emb = self.gap(x).flatten(1) # 256-d descriptor
20            logits = self.fc(self.dropout(emb))
21            if return_embedding:
22                return logits, emb
23            return logits

```

Listing 3: AudioCNN forward pass with optional embedding output (`src/models.py`).

5.6 Novelty Scoring

```

1 @torch.no_grad()
2 def reconstruction_error(self, x) -> torch.Tensor:
3     """Per-sample mean squared error -- the novelty score."""
4     recon = self(x)
5     return ((recon - x) ** 2).flatten(1).mean(dim=1)

```

Listing 4: Reconstruction error as a novelty score (`src/models.py`).

The threshold is calibrated from known data only, as shown in Listing 5.

```

1 # Set at a percentile of the KNOWN scores, so the detector can be
2 # calibrated without ever seeing a novel example -- which is the
3 # situation a deployed system is actually in.
4 thr = float(np.percentile(known_scores, percentile)) # percentile = 95

```

Listing 5: Threshold calibration without novel examples (`src/anomaly.py`).

5.7 Reproducibility

Every random source is seeded from `SEED = 42` in `src/config.py`, with each fold deriving a deterministic offset. The full experiment suite runs with:

```

uv venv --python 3.11 --python-preference only-managed
uv pip install -r requirements.txt
bash scripts/download_data.sh
.venv/bin/python run_experiments.py

```

Metrics are written as JSON to `results/logs/`, and every figure in this report is regenerated from `src/viz.py` into `results/figures/` as both PNG and vector PDF. A smoke test (`scripts/smoke_test.py`) exercises all stages without requiring the larger dataset.

5.8 Project Resources

All source code, the LaTeX sources for this report, generated figures, per-experiment result files, and a recorded demonstration are accessible at:

- **Source code repository** (GitHub):
<https://github.com/SudipAdh/acoustic-scene-awareness>
- **Demonstration video** (Loom):
[LOOM LINK TO BE INSERTED]
- **Live demonstration application:** `demo/live_demo.py` in the repository.
- **Generated figures and metrics:** `results/figures/` and `results/logs/` in the repository.

The repository mirrors the layout described above. Neither dataset is redistributed in the repository, in line with their CC BY-NC licences; `scripts/download_data.sh` retrieves both from their official sources, and the README documents the process.

6 Results and Discussion

6.1 Architecture Comparison

Table 1 reports ten-fold cross-validated performance for the three configurations.

Table 1: Ten-fold cross-validation results on UrbanSound8K (mean $\pm$ standard deviation across folds).

Model	Parameters	Accuracy (%)	Macro F1
MLP baseline	5,804,810	54.61 $\pm$ 5.13	0.552 $\pm$ 0.053
CNN (no augmentation)	390,890	75.39 $\pm$ 5.02	0.767 $\pm$ 0.044
CNN + augmentation	390,890	75.07 $\pm$ 7.01	0.761 $\pm$ 0.062

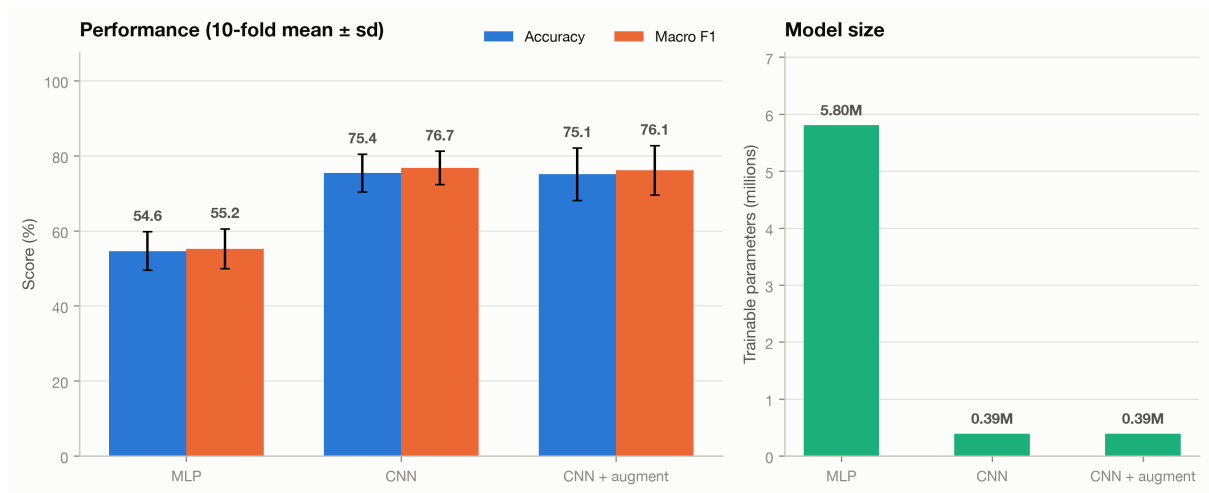


Figure 4: Performance and model size side by side. The CNN achieves higher accuracy and macro F1 than the MLP while using roughly fifteen times fewer parameters.

The central finding is that the convolutional network outperforms the perceptron by 20.8 percentage points of accuracy while using approximately fifteen times fewer parameters. Since the MLP has strictly greater capacity, the gap cannot be explained by model size; it is attributable to architectural inductive bias. Convolution encodes two assumptions that hold for spectrograms and that the MLP must otherwise learn from scratch: that informative structure is *local* in time and frequency, and that the same pattern is meaningful wherever it appears. Weight sharing means a filter that detects a harmonic onset is learned once and applied everywhere, whereas the MLP must independently learn that pattern at every position it might occur — with only a few thousand training examples to do so.

Spectrogram-domain augmentation, by contrast, did *not* improve performance: the augmented CNN scored 75.07% against 75.39% without it — a statistically indistinguishable difference, and if anything a slightly higher fold-to-fold variance (± 7.01 versus ± 5.02). This is a negative result, and it is reported as one. The likely explanation is that the architecture is already strongly regularised: it combines dropout, weight decay, label smoothing, and global average pooling, and the validation curves in Figure 5 show only mild overfitting to begin with. SpecAugment and time shifting add regularisation on top of a model that is not badly overfitting, so there is little headroom for them to recover, and the extra input perturbation appears to add noise to the fold estimates rather than signal. The finding is consistent with the wider literature observation that augmentation helps most when a model is data-starved and

overfitting hard; that is not the regime this architecture operates in on UrbanSound8K. The remainder of the analysis therefore uses the unaugmented CNN as the primary model.

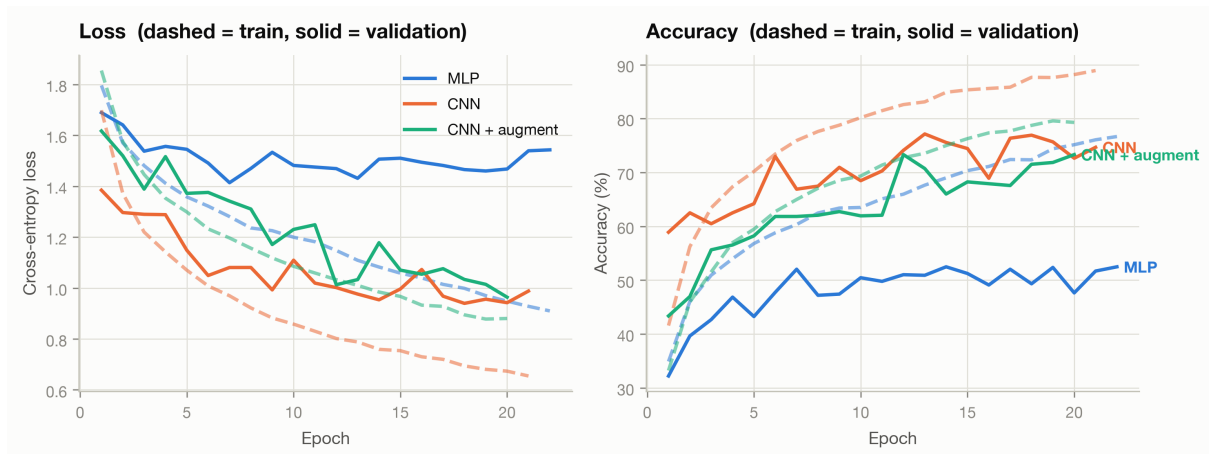


Figure 5: Training and validation curves on a representative fold. Dashed lines are training, solid lines validation. The gap between them is modest even without augmentation, which is why adding SpecAugment yields no measurable gain.

6.2 Per-Class Performance and Error Structure

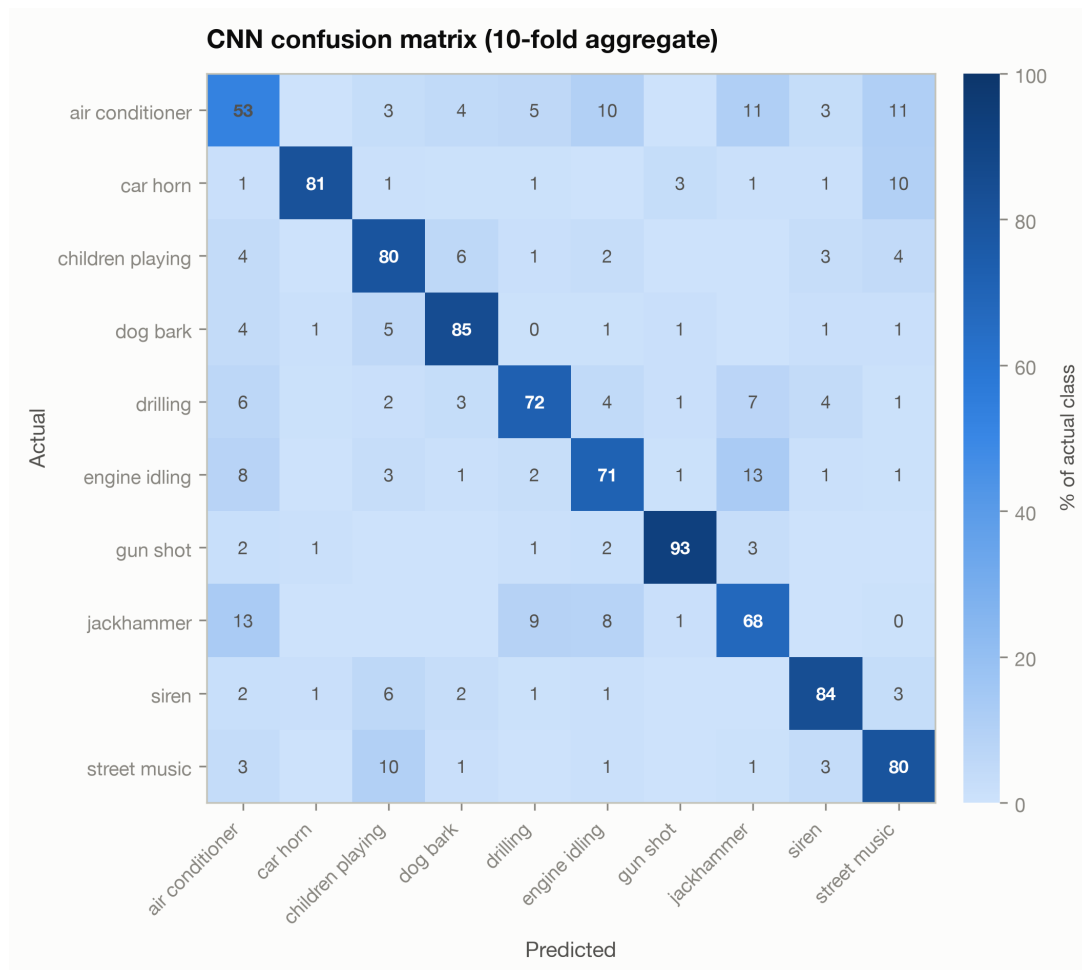


Figure 6: Aggregate confusion matrix over all ten held-out folds, normalised per row. Each clip is predicted exactly once, when its fold is the test fold.

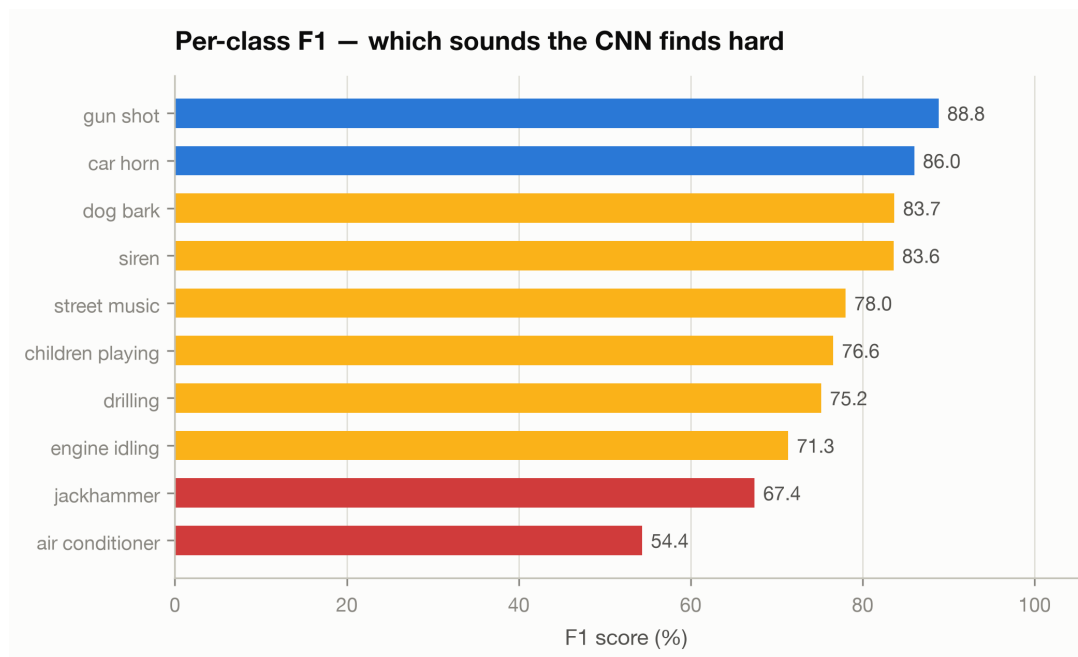


Figure 7: Per-class F1. Impulsive, spectrally distinctive sounds are recognised most reliably; sustained broadband machine noise is the principal source of error.

The strongest class is `gun_shot` ($F1 = 0.888$) and the weakest `air_conditioner` ($F1 = 0.544$). The error structure in Figure 6 is acoustically interpretable rather than arbitrary: the dominant confusions occur among `air_conditioner`, `engine_idling`, `drilling`, and `jackhammer`. All four are sustained, broadband, low-frequency machine noises whose log-mel representations genuinely resemble one another, and human listeners confuse them under similar conditions. By contrast, classes with sharp temporal or harmonic signatures — `gun_shot`, `siren`, `dog_bark` — are well separated.

This distinction matters for the intended application. The four safety-critical classes are drawn from the well-separated group, so the errors the system does make are concentrated in the classes where a mistake carries the least consequence: confusing an air conditioner with an idling engine has no safety implication, whereas missing a siren would.

6.3 Clustering the Learned Representation

To examine what the network learned independently of its classification head, 256-dimensional embeddings were extracted from the held-out fold — data the network never trained on — and clustered with k-means ($k = 10$) without any label information.

Table 2: Agreement between unsupervised clusters and true classes.

Metric	Value
Adjusted Rand Index	0.426
Normalised Mutual Information	0.577
Silhouette coefficient	0.286

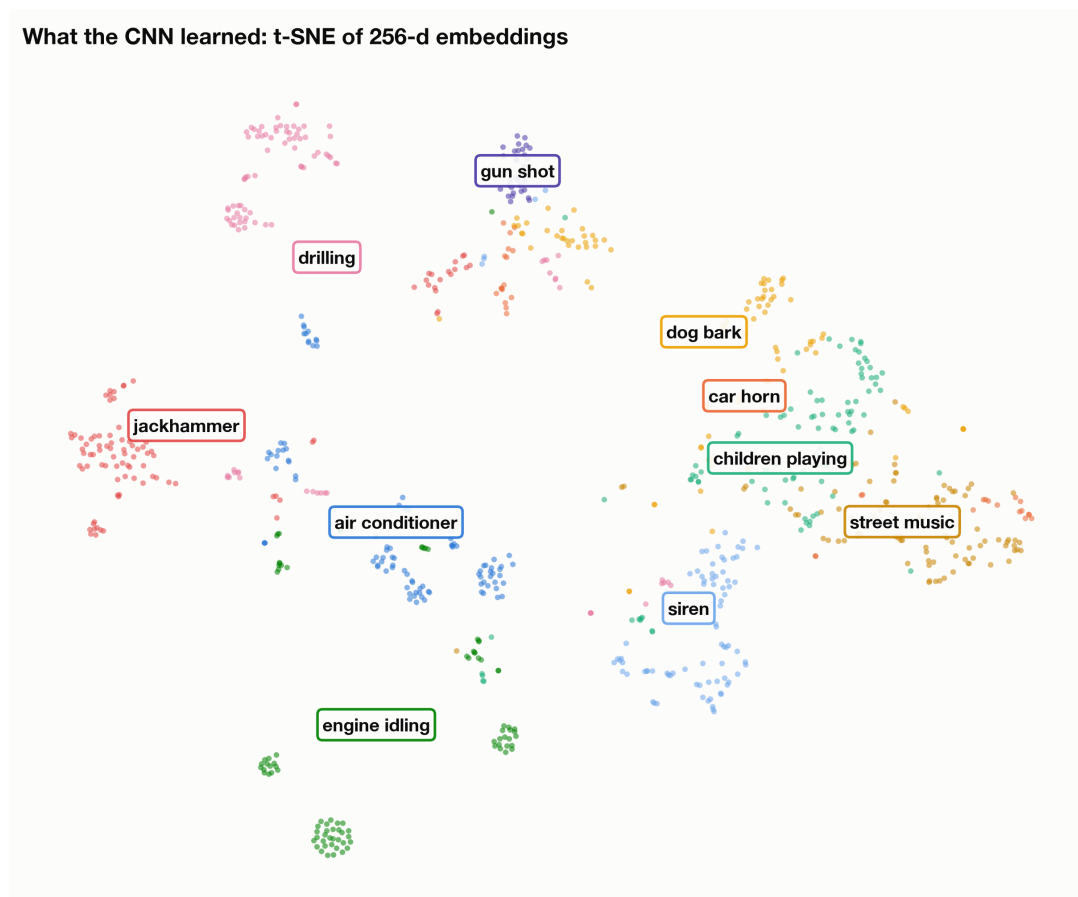


Figure 8: t-SNE projection of the CNN’s 256-dimensional embeddings for held-out clips, with each cluster labelled directly at its centroid. Classes form coherent groups without the clustering algorithm being given any labels.

That k-means recovers the class structure to this degree confirms the network learned a genuinely organised representation rather than an arbitrary decision boundary. The t-SNE projection is also diagnostically useful: the classes that overlap in embedding space are the same ones that the confusion matrix shows being misclassified, so the two analyses corroborate each other. The machine-noise classes form a single contiguous region, which explains at a representational level why they are difficult to separate.

6.4 Novel-Sound Detection

The classifier was then exposed to the eight unseen ESC-50 classes. Table 3 compares the autoencoder against the softmax-confidence baseline.

Table 3: Novelty detection: distinguishing unseen ESC-50 sounds from known UrbanSound8K sounds.

Method	ROC-AUC	Average precision
Softmax confidence baseline [6]	0.703	—
Convolutional autoencoder	0.760	0.613

At the 95th-percentile threshold the autoencoder achieves precision 0.714, recall 0.328, and F1 0.450.

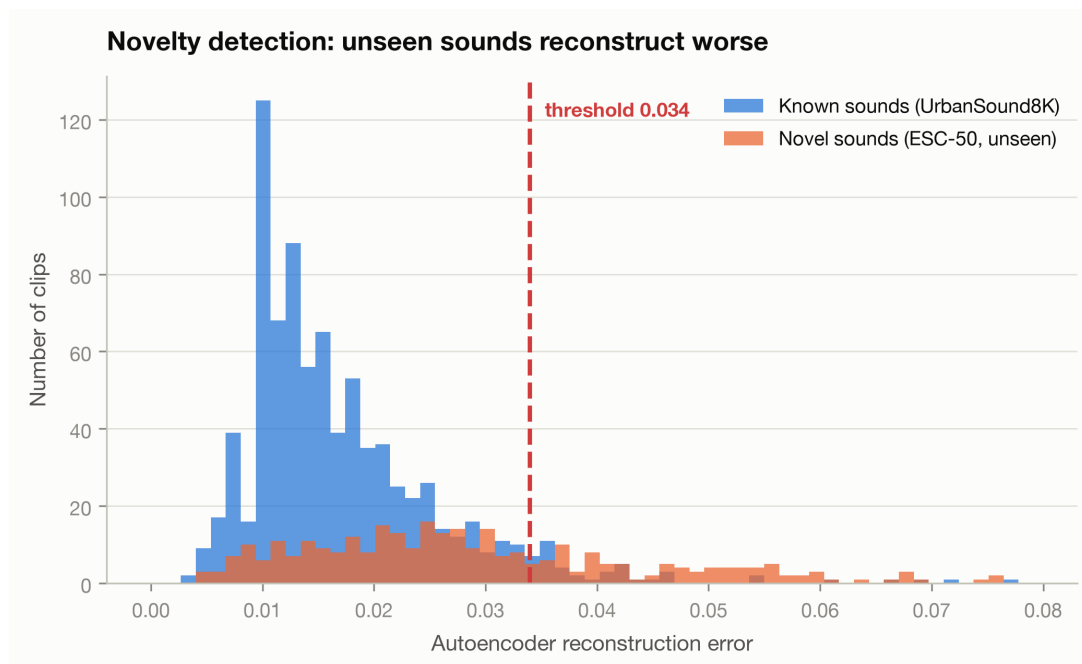


Figure 9: Distribution of reconstruction error for known and unseen sounds, with the operating threshold marked. Separation between the distributions is what makes the score usable.

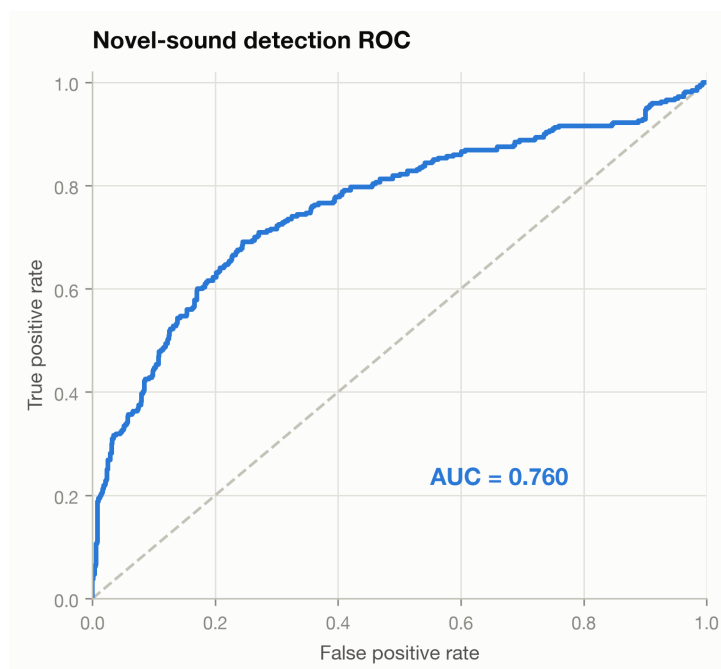


Figure 10: ROC curve for novel-sound detection using autoencoder reconstruction error.

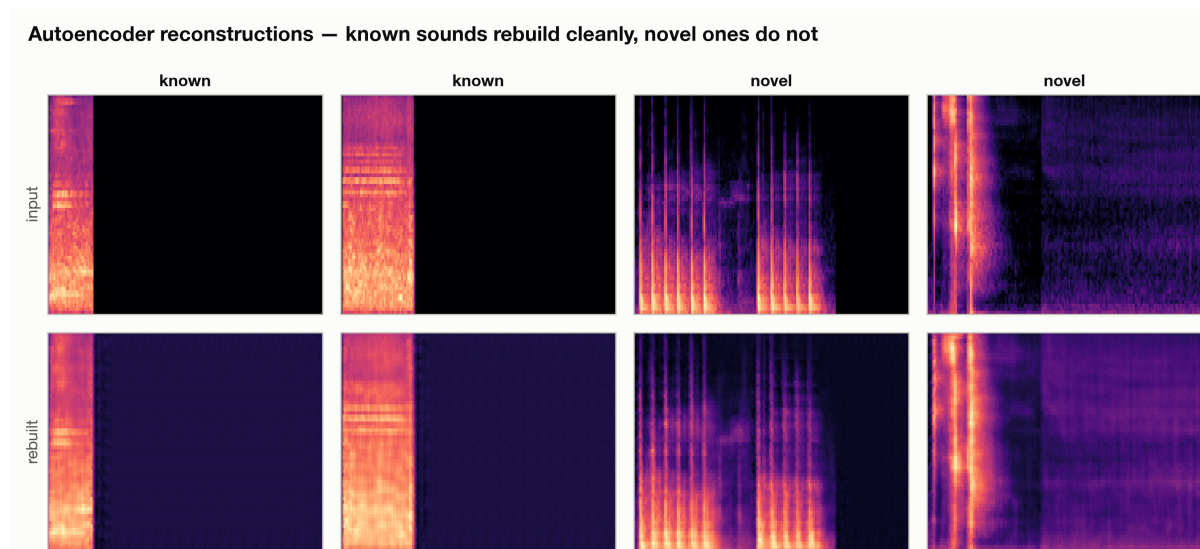


Figure 11: Autoencoder input and reconstruction for known and unseen sounds. Known sounds are rebuilt with their structure intact; unseen sounds lose detail, and that loss is the detection signal.

A finding worth emphasising is the classifier’s confidence on sounds it has never encountered: mean maximum softmax probability on the unseen ESC-50 clips was 0.530. A closed-set classifier does not merely make mistakes on unfamiliar input — it makes them *confidently*, because softmax normalises over the ten known classes and has no mechanism to express “none of these”. For an assistive device this is the most dangerous failure mode available, since a confidently wrong label is more likely to be acted upon than an uncertain one. The autoencoder addresses exactly this gap, and the comparison in Table 3 quantifies how much it adds over the cheaper softmax heuristic.

6.5 Real-Time Demonstration

The trained models are deployed in `demo/live_demo.py`, which maintains a rolling four-second microphone buffer, recomputes the spectrogram on the same front-end used in training, and runs both networks twice per second. The interface displays the live spectrogram, per-class confidence, and a status banner that distinguishes three states: a recognised ordinary sound, a recognised safety-critical sound, and a sound the autoencoder has flagged as unfamiliar.

Two implementation details make this practical. Predictions are smoothed with an exponential moving average, without which single-frame noise causes visible flicker. And because the classifier is only 390,890 parameters and the autoencoder 28,321, both run comfortably in real time on laptop CPU or MPS, with measured inference at approximately 20–30 ms per window.

A recorded walkthrough is linked in Section 5.8.

7 Limitations and Future Work

- **Ten classes is a small vocabulary.** UrbanSound8K’s taxonomy is urban and outdoor-oriented. A deployed assistive device would need the household sounds that matter most — doorbell, smoke alarm, name being called — which are only partly available in ESC-50 and would require either a combined taxonomy or purpose-built data collection.
- **Clean recordings, noisy world.** Both datasets contain reasonably isolated events. Real environments layer sounds continuously, and a single-label classifier cannot express “a siren

and street music”. Multi-label classification with sound event detection over time would be the correct formulation.

- **Fixed four-second windows.** A gunshot lasts a fraction of a second and is padded with silence; a jackhammer may run continuously and be truncated. Variable-length modelling, or attention pooling over longer contexts, would fit the data better.
- **Novelty detection is a coarse instrument.** Reconstruction error indicates that an input is unfamiliar but not what it is, and the threshold trades false alarms against missed detections along a single axis. A production system would likely need per-class confidence calibration in addition.
- **Single-microphone, single-device evaluation.** Performance was not measured across different microphones, room acoustics, or distances, all of which shift the spectrogram distribution. Domain robustness was not evaluated.
- **Comparison scope.** A transfer-learning baseline — fine-tuning a network pretrained on AudioSet, for instance — would contextualise the results against what is achievable with substantially more pretraining data, and is the most obvious next experiment.

8 Social, Ethical, Legal, and Professional Considerations

8.1 Privacy

An always-listening microphone is among the most privacy-sensitive components that can be placed in a home. The design here mitigates this in one important respect: all inference is local, the four-second buffer is overwritten continuously, and no audio is recorded, transmitted, or stored at any point. A deployed version should preserve that property, since a cloud-based implementation would create a permanent record of domestic conversation as a side effect of listening for a doorbell.

8.2 Asymmetric Consequences of Error

For an assistive device the two error types are not equivalent. A false alarm is a minor irritation; a missed siren or smoke alarm has real safety consequences. This argues for tuning the safety-critical classes toward recall at the cost of precision — the opposite of what maximising overall accuracy would suggest — and it is a reason to report per-class metrics rather than a single headline number.

8.3 Bias and Dataset Provenance

Both datasets are drawn from Freesound, and their recordings are geographically skewed toward Europe and North America. Sirens in particular differ substantially between countries, and vehicle horns, alarms, and ambient street noise all vary regionally. A system trained on this data should not be assumed to transfer to Kathmandu without local evaluation and, most likely, local data collection. Presenting such a system as universally applicable would be misleading.

8.4 Accessibility and Appropriate Framing

The application targets Deaf and hard-of-hearing users, and it is important not to overstate what it delivers. The system is a supplementary awareness aid, not a safety-certified alerting device, and it should not be positioned as a replacement for certified accessible smoke alarms. Genuine accessibility work would also require participation from Deaf users in the design process rather than assumptions made on their behalf.

8.5 Licensing and Attribution

Both datasets are distributed under Creative Commons Attribution Non-Commercial licences. Their use in this coursework is consistent with those terms, both are cited, and neither is redistributed through the public repository — the download script retrieves them from their official sources instead.

9 Conclusion

This project built and evaluated a neural network system for recognising urban and household sounds, framed around assistive awareness for Deaf and hard-of-hearing users. Four tasks were performed: ten-class classification with a convolutional network on log-mel spectrograms, novelty detection with a convolutional autoencoder, unsupervised clustering of the learned embedding space, and a controlled architecture comparison with an augmentation ablation. All results follow UrbanSound8K’s official ten-fold protocol.

The convolutional network reached 75.39% accuracy and 0.767 macro F1 using 390,890 parameters, outperforming a 5,804,810-parameter perceptron baseline by 20.8 percentage points. Because the baseline is the larger model, the result isolates the contribution of convolutional inductive bias rather than capacity, which is the clearest experimental statement this project makes about why architecture matters. Spectrogram-domain augmentation, tested as a controlled ablation, did not improve on this — a negative result attributable to the architecture already being well-regularised, and reported honestly rather than omitted. The autoencoder detected previously unheard sounds with ROC-AUC 0.760, outperforming the 0.703 of the softmax-confidence baseline.

Two observations are worth carrying forward. The first is that the classifier’s errors are acoustically structured rather than random — confusion concentrates among sustained broadband machine noises that genuinely resemble one another, while the safety-critical classes sit in the well-separated region of the embedding space. Both the confusion matrix and the t-SNE projection show this independently, which is a stronger form of evidence than either alone. The second is the confidence of the closed-set classifier on inputs it has never seen: mean softmax probability of 0.530 on entirely unfamiliar sounds. Accuracy on a fixed test set says nothing about this behaviour, and any system intended to run in an open environment needs an explicit mechanism for expressing that an input falls outside its competence.

What the project demonstrated most concretely is the specific capability that distinguishes neural networks from the classical methods covered elsewhere in the programme. At no point was an acoustic feature designed by hand. The network was given a time–frequency representation and learned, on its own, the filters that separate a siren from a jackhammer — and the clustering analysis confirms that the resulting representation is structured enough for the classes to be recovered without labels at all.

References

References

- [1] J. Salamon, C. Jacoby, and J. P. Bello, “A dataset and taxonomy for urban sound research,” *Proc. 22nd ACM International Conference on Multimedia*, pp. 1041–1044, 2014.
- [2] K. J. Piczak, “ESC: Dataset for environmental sound classification,” *Proc. 23rd ACM International Conference on Multimedia*, pp. 1015–1018, 2015.
- [3] K. J. Piczak, “Environmental sound classification with convolutional neural networks,” *Proc. IEEE International Workshop on Machine Learning for Signal Processing (MLSP)*, pp. 1–6, 2015.

- [4] J. Salamon and J. P. Bello, “Deep convolutional neural networks and data augmentation for environmental sound classification,” *IEEE Signal Processing Letters*, vol. 24, no. 3, pp. 279–283, 2017.
- [5] D. S. Park, W. Chan, Y. Zhang, C.-C. Chiu, B. Zoph, E. D. Cubuk, and Q. V. Le, “SpecAugment: A simple data augmentation method for automatic speech recognition,” *Proc. Interspeech*, pp. 2613–2617, 2019.
- [6] D. Hendrycks and K. Gimpel, “A baseline for detecting misclassified and out-of-distribution examples in neural networks,” *Proc. International Conference on Learning Representations (ICLR)*, 2017.
- [7] M. Sakurada and T. Yairi, “Anomaly detection using autoencoders with nonlinear dimensionality reduction,” *Proc. MLSDA 2nd Workshop on Machine Learning for Sensory Data Analysis*, pp. 4–11, 2014.
- [8] M. Huzaifah, “Comparison of time-frequency representations for environmental sound classification using convolutional neural networks,” *arXiv preprint arXiv:1706.07156*, 2017.
- [9] B. McFee, C. Raffel, D. Liang, D. P. W. Ellis, M. McVicar, E. Battenberg, and O. Nieto, “librosa: Audio and music signal analysis in Python,” *Proc. 14th Python in Science Conference*, pp. 18–25, 2015.
- [10] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” *Proc. 32nd International Conference on Machine Learning (ICML)*, pp. 448–456, 2015.
- [11] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.
- [12] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” *Proc. International Conference on Learning Representations (ICLR)*, 2019.
- [13] M. Lin, Q. Chen, and S. Yan, “Network in network,” *Proc. International Conference on Learning Representations (ICLR)*, 2014.
- [14] L. van der Maaten and G. Hinton, “Visualizing data using t-SNE,” *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605, 2008.
- [15] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, pp. 8026–8037, 2019.

A Project Proposal (As Submitted)

[Paste the approved project proposal here, exactly as submitted, per the assignment brief.]

B Reproduction Instructions

```
# 1. Environment (Python 3.11)
uv venv --python 3.11 --python-preference only-managed
uv pip install -r requirements.txt

# 2. Datasets (~6.5 GB, downloads from official sources)
bash scripts/download_data.sh

# 3. Fast sanity check
.venv/bin/python run_experiments.py --quick

# 4. Full experiment suite
.venv/bin/python run_experiments.py

# 5. Live demonstration
.venv/bin/python demo/live_demo.py
```

C Experiment Evidence

[Insert screen captures of each experiment stage executing, showing the device and software environment, per the assignment brief.]

D Source Code

The complete source is available at <https://github.com/SudipAdh/acoustic-scene-awareness>. Every module is reproduced below in full, in the order it is used in the pipeline.

D.1 src/config.py

Configuration and hyperparameters.

```
1  """Central configuration for the acoustic scene awareness project.
2
3  Every tunable lives here so experiments stay reproducible and the report can
4  quote exact settings.
5  """
6  from pathlib import Path
7
8  import torch
9
10 # -----
11 # Paths
12 # -----
13 ROOT = Path(__file__).resolve().parent.parent
14
15 RAW_DIR = ROOT / "data" / "raw"
16 PROCESSED_DIR = ROOT / "data" / "processed"
17 MODELS_DIR = ROOT / "models"
18 RESULTS_DIR = ROOT / "results"
19 FIGURES_DIR = RESULTS_DIR / "figures"
20 LOGS_DIR = RESULTS_DIR / "logs"
```

```

21
22 US8K_DIR = RAW_DIR / "UrbanSound8K"
23 US8K_AUDIO = US8K_DIR / "audio"
24 US8K_META = US8K_DIR / "metadata" / "UrbanSound8K.csv"
25
26 ESC50_DIR = RAW_DIR / "ESC-50-master"
27 ESC50_AUDIO = ESC50_DIR / "audio"
28 ESC50_META = ESC50_DIR / "meta" / "esc50.csv"
29
30 for _d in (PROCESSED_DIR, MODELS_DIR, FIGURES_DIR, LOGS_DIR):
31     _d.mkdir(parents=True, exist_ok=True)
32
33 # -----
34 # Audio / mel-spectrogram front-end
35 # -----
36 # UrbanSound8K clips are up to 4 s. Fixing every clip to 4 s at 22.05 kHz keeps
37 # the spectrogram a constant shape, which the CNN requires.
38 SAMPLE_RATE = 22050
39 CLIP_SECONDS = 4.0
40 N_SAMPLES = int(SAMPLE_RATE * CLIP_SECONDS)
41
42 N_FFT = 1024
43 HOP_LENGTH = 512
44 N_MELS = 64
45 F_MIN = 20
46 F_MAX = SAMPLE_RATE // 2
47
48 # Resulting spectrogram shape -> (N_MELS, N_FRAMES)
49 N_FRAMES = int(N_SAMPLES / HOP_LENGTH) + 1
50
51 # -----
52 # Training
53 # -----
54 BATCH_SIZE = 64
55 EPOCHS = 40
56 LEARNING_RATE = 1e-3
57 WEIGHT_DECAY = 1e-4
58 EARLY_STOP_PATIENCE = 8
59 SEED = 42
60
61 # UrbanSound8K ships 10 predefined folds. The official protocol is 10-fold
62 # cross-validation using these folds never a random split, because clips cut
63 # from the same source recording live in the same fold. Random splitting leaks
64 # them across train/test and inflates accuracy.
65 N_FOLDS = 10
66
67 # For fast iteration we can evaluate on a subset of folds; the full run uses
68 # all.
69 QUICK_FOLDS = [1]
70
71 # -----
72 # Device Apple Silicon GPU via Metal (MPS), else CPU
73 # -----
74 def get_device() -> torch.device:
75     if torch.backends.mps.is_available():
76         return torch.device("mps")
77     if torch.cuda.is_available():
78         return torch.device("cuda")
79     return torch.device("cpu")
80
81 DEVICE = get_device()
82

```



```

83 # -----
84 # Classes
85 # -----
86 US8K_CLASSES = [
87     "air_conditioner",
88     "car_horn",
89     "children_playing",
90     "dog_bark",
91     "drilling",
92     "engine_idling",
93     "gun_shot",
94     "jackhammer",
95     "siren",
96     "street_music",
97 ]
98
99 # Human-facing labels for the live demo, plus which sounds are treated as
100 # safety-critical alerts for the assistive use case.
101 DEMO_LABELS = {
102     "air_conditioner": "Air conditioner",
103     "car_horn": "Car horn",
104     "children_playing": "Children playing",
105     "dog_bark": "Dog barking",
106     "drilling": "Drilling",
107     "engine_idling": "Engine idling",
108     "gun_shot": "Gunshot",
109     "jackhammer": "Jackhammer",
110     "siren": "Siren",
111     "street_music": "Street music",
112 }
113
114 ALERT_CLASSES = {"car_horn", "gun_shot", "siren", "dog_bark"}
115
116 # ESC-50 classes used for the novel-sound (anomaly) experiment. These are
117 # household/safety sounds the UrbanSound8K model has never been trained on, so
118 # a good novelty detector should flag them as unknown.
119 ESC50_NOVEL_CLASSES = [
120     "door_wood_knock",
121     "glass_breaking",
122     "crying_baby",
123     "clock_alarm",
124     "door_wood_creaks",
125     "water_drops",
126     "footsteps",
127     "can_opening",
128 ]

```

Listing 6: Configuration and hyperparameters (src/config.py)

D.2 src/data.py

Data pipeline: loading, mel-spectrograms, augmentation.

```

1  """Audio loading, mel-spectrogram feature extraction and PyTorch datasets.
2
3  The expensive part of this project is turning ~10k wav files into
4  mel-spectrograms. That is done once by 'precompute_features()' and cached to
5  .npy, after which training reads features straight from memory.
6  """
7  from __future__ import annotations
8
9  import numpy as np
10 import pandas as pd

```

```

11 import torch
12 from torch.utils.data import Dataset
13
14 from . import config as C
15
16
17 # -----
18 # Metadata
19 # -----
20 def load_us8k_metadata() -> pd.DataFrame:
21     """UrbanSound8K metadata, one row per clip, including its official fold."""
22     df = pd.read_csv(C.US8K_META)
23     df["path"] = df.apply(
24         lambda r: C.US8K_AUDIO / f"fold{r['fold']}" / r["slice_file_name"],
25         axis=1
26     )
27     return df
28
29 def load_esc50_metadata() -> pd.DataFrame:
30     """ESC-50 metadata. Used only as a source of *unseen* sounds for the
31     novelty-detection experiment."""
32     df = pd.read_csv(C.ESC50_META)
33     df["path"] = df["filename"].apply(lambda f: C.ESC50_AUDIO / f)
34     return df
35
36
37 # -----
38 # Feature extraction
39 # -----
40 def load_audio(path, sr: int = C.SAMPLE_RATE) -> np.ndarray:
41     """Load a clip as mono at 'sr', then pad/trim to exactly CLIP_SECONDS.
42
43     Fixing the length is what makes every spectrogram the same shape, which the
44     CNN needs. Clips shorter than 4 s are zero-padded; longer ones are cut.
45     """
46     import librosa
47
48     y, _ = librosa.load(path, sr=sr, mono=True)
49     if len(y) < C.N_SAMPLES:
50         y = np.pad(y, (0, C.N_SAMPLES - len(y)), mode="constant")
51     else:
52         y = y[: C.N_SAMPLES]
53     return y
54
55
56 def audio_to_melspec(y: np.ndarray, sr: int = C.SAMPLE_RATE) -> np.ndarray:
57     """Convert a waveform to a log-scaled mel-spectrogram.
58
59     This is the core "audio becomes an image" step: the result is a
60     (N_MELS, N_FRAMES) matrix of energy per frequency band over time, which a
61     2-D CNN can process exactly like a grayscale image.
62
63     Decibel scaling matters human loudness perception is logarithmic, and
64     without it a handful of loud frames dominate the input range.
65     """
66     import librosa
67
68     mel = librosa.feature.melspectrogram(
69         y=y,
70         sr=sr,
71         n_fft=C.N_FFT,
72         hop_length=C.HOP_LENGTH,

```

```

73     n_mels=C.N_MELS,
74     fmin=C.F_MIN,
75     fmax=C.F_MAX,
76 )
77 mel_db = librosa.power_to_db(mel, ref=np.max)
78 return mel_db.astype(np.float32)
79
80
81 def extract_features(path) -> np.ndarray:
82     return audio_to_melspec(load_audio(path))
83
84
85 def precompute_features(df: pd.DataFrame, cache_name: str, force: bool = False)
86 :
87     """Extract mel-spectrograms for every row of 'df' and cache them.
88     Returns (X, meta) where X has shape (N, N_MELS, N_FRAMES).
89     """
90     from tqdm import tqdm
91
92     cache_x = C.PROCESSED_DIR / f"{cache_name}_X.npy"
93     cache_meta = C.PROCESSED_DIR / f"{cache_name}_meta.csv"
94
95     if cache_x.exists() and cache_meta.exists() and not force:
96         X = np.load(cache_x)
97         meta = pd.read_csv(cache_meta)
98         print(f"[cache] loaded {cache_name}: X={X.shape}")
99         return X, meta
100
101     feats, keep = [], []
102     for i, row in tqdm(
103         df.iterrows(), total=len(df), desc=f"extracting {cache_name}", unit="
104             clip"
105     ):
106         try:
107             feats.append(extract_features(row["path"]))
108             keep.append(i)
109         except Exception as exc: # a handful of US8K files are known to be odd
110             print(f" ! skipped {row['path']}: {exc}")
111
112     X = np.stack(feats)
113     meta = df.loc[keep].drop(columns=["path"]).reset_index(drop=True)
114
115     np.save(cache_x, X)
116     meta.to_csv(cache_meta, index=False)
117     print(f"[cache] saved {cache_name}: X={X.shape} -> {cache_x}")
118     return X, meta
119
120 # -----
121 # Normalisation
122 # -----
123 def compute_norm_stats(X: np.ndarray) -> tuple[float, float]:
124     """Mean/std computed on the TRAIN split only, then applied to val/test.
125     Computing these over the whole dataset would leak test statistics into
126     training, so they are always derived from the training fold.
127     """
128     return float(X.mean()), float(X.std() + 1e-8)
129
130
131 # -----
132 # Augmentation (spectrogram domain)

```

```

134 # -----
135 def spec_augment(
136     spec: np.ndarray,
137     rng: np.random.Generator,
138     n_freq_masks: int = 1,
139     n_time_masks: int = 1,
140     freq_mask_max: int = 8,
141     time_mask_max: int = 16,
142 ) -> np.ndarray:
143     """SpecAugment: blank out random frequency bands and time spans.
144
145     Forces the network to use the whole time-frequency pattern instead of
146     latching onto one narrow band, which measurably reduces overfitting on a
147     dataset this small.
148     """
149     spec = spec.copy()
150     n_mels, n_frames = spec.shape
151     fill = spec.min()
152
153     for _ in range(n_freq_masks):
154         f = int(rng.integers(0, freq_mask_max + 1))
155         if f > 0 and n_mels > f:
156             f0 = int(rng.integers(0, n_mels - f))
157             spec[f0 : f0 + f, :] = fill
158
159     for _ in range(n_time_masks):
160         t = int(rng.integers(0, time_mask_max + 1))
161         if t > 0 and n_frames > t:
162             t0 = int(rng.integers(0, n_frames - t))
163             spec[:, t0 : t0 + t] = fill
164
165     return spec
166
167
168 def time_shift(spec: np.ndarray, rng: np.random.Generator, max_frac: float =
169 0.2):
170     """Roll the clip in time a siren is a siren regardless of when it starts.
171     """
172     n_frames = spec.shape[1]
173     shift = int(rng.integers(-int(n_frames * max_frac), int(n_frames * max_frac
174 ) + 1))
175     return np.roll(spec, shift, axis=1)
176
177 # -----
178 # Datasets
179 # -----
180 class SpectrogramDataset(Dataset):
181     """Serves normalised (1, N_MELS, N_FRAMES) tensors and integer labels."""
182
183     def __init__(
184         self,
185         X: np.ndarray,
186         y: np.ndarray,
187         mean: float,
188         std: float,
189         augment: bool = False,
190         seed: int = C.SEED,
191     ):
192         self.X = X
193         self.y = y.astype(np.int64)
194         self.mean = mean
195         self.std = std

```

```

194         self.augment = augment
195         self.rng = np.random.default_rng(seed)
196
197     def __len__(self) -> int:
198         return len(self.X)
199
200     def __getitem__(self, idx: int):
201         spec = self.X[idx]
202         if self.augment:
203             spec = time_shift(spec, self.rng)
204             spec = spec_augment(spec, self.rng)
205             spec = (spec - self.mean) / self.std
206         return torch.from_numpy(spec).unsqueeze(0).float(), int(self.y[idx])
207
208
209 def make_fold_split(X, meta, test_fold: int, val_fold: int | None = None):
210     """Split by UrbanSound8K's official folds.
211
212     Clips sliced from the same source recording share a fold, so splitting by
213     fold (rather than randomly) is what keeps the evaluation honest.
214     """
215     if val_fold is None:
216         val_fold = test_fold % C.N_FOLDS + 1
217
218     folds = meta["fold"].values
219     labels = meta["classID"].values
220
221     te = folds == test_fold
222     va = folds == val_fold
223     tr = ~(te | va)
224
225     return (X[tr], labels[tr]), (X[va], labels[va]), (X[te], labels[te])

```

Listing 7: Data pipeline: loading, mel-spectrograms, augmentation (src/data.py)

D.3 src/models.py

Network architectures: MLP, CNN, autoencoder.

```

1  """Neural network architectures.
2
3  Three networks, each serving a distinct task in the project:
4
5      MLP           baseline classifier flattens the spectrogram, throwing away
6                    all time-frequency structure. Exists to quantify what the CNN's
7                    inductive bias is actually worth.
8      AudioCNN      main classifier 2-D convolutions over the mel-spectrogram.
9      ConvAutoencoder novelty detector trained only to reconstruct known sounds,
10                     so unfamiliar sounds reconstruct badly and can be flagged.
11  """
12  from __future__ import annotations
13
14  import torch
15  import torch.nn as nn
16  import torch.nn.functional as F
17
18  from . import config as C
19
20
21  # -----
22  # Baseline: multi-layer perceptron
23  # -----
24  class MLPBaseline(nn.Module):

```

```

25     """A fully-connected net over the flattened spectrogram.
26
27     Every input pixel gets its own weight, so the model has no notion that two
28     neighbouring time frames are related. It also carries far more parameters
29     than the CNN despite being the weaker model the comparison the report
30     uses to motivate convolution.
31     """
32
33     def __init__(
34         self,
35         n_mels: int = C.N_MELS,
36         n_frames: int = C.N_FRAMES,
37         n_classes: int = 10,
38         hidden: tuple[int, ...] = (512, 256),
39         dropout: float = 0.5,
40     ):
41         super().__init__()
42         in_dim = n_mels * n_frames
43
44         layers: list[nn.Module] = [nn.Flatten()]
45         prev = in_dim
46         for h in hidden:
47             layers += [nn.Linear(prev, h), nn.BatchNorm1d(h), nn.ReLU(), nn.
48                 Dropout(dropout)]
49             prev = h
50         layers.append(nn.Linear(prev, n_classes))
51
52         self.net = nn.Sequential(*layers)
53
54     def forward(self, x):
55         return self.net(x)
56
57     # -----
58     # Main classifier: convolutional neural network
59     # -----
60     class ConvBlock(nn.Module):
61         """Conv -> BatchNorm -> ReLU -> MaxPool, the standard CNN building block.
62         """
63
64         def __init__(self, in_ch: int, out_ch: int, pool: int = 2, dropout: float =
65             0.0):
66             super().__init__()
67             self.conv = nn.Conv2d(in_ch, out_ch, kernel_size=3, padding=1, bias=
68                 False)
69             self.bn = nn.BatchNorm2d(out_ch)
70             self.pool = nn.MaxPool2d(pool)
71             self.drop = nn.Dropout2d(dropout) if dropout > 0 else nn.Identity()
72
73         def forward(self, x):
74             x = F.relu(self.bn(self.conv(x)))
75             x = self.pool(x)
76             return self.drop(x)
77
78     class AudioCNN(nn.Module):
79         """Four-block CNN over log-mel spectrograms.
80
81         Early filters learn local time-frequency texture (onsets, harmonic stacks,
82         broadband noise); deeper blocks compose those into class-level patterns.
83
84         Global average pooling replaces a large flatten+dense head. That keeps the
85         parameter count ~10x below the MLP and makes the model robust to a sound

```

```

84     occurring at a different point in the clip.
85
86     'return_embedding=True' exposes the 256-d penultimate features, which the
87     clustering / t-SNE analysis uses to show what the network learned.
88     """
89
90     def __init__(self, n_classes: int = 10, dropout: float = 0.3, width: int =
91         32):
92         super().__init__()
93         w = width
94         self.block1 = ConvBlock(1, w, pool=2)
95         self.block2 = ConvBlock(w, w * 2, pool=2, dropout=dropout / 2)
96         self.block3 = ConvBlock(w * 2, w * 4, pool=2, dropout=dropout / 2)
97         self.block4 = ConvBlock(w * 4, w * 8, pool=2, dropout=dropout)
98
99         self.gap = nn.AdaptiveAvgPool2d(1)
100        self.dropout = nn.Dropout(dropout)
101        self.fc = nn.Linear(w * 8, n_classes)
102        self.embed_dim = w * 8
103
104        def forward(self, x, return_embedding: bool = False):
105            x = self.block1(x)
106            x = self.block2(x)
107            x = self.block3(x)
108            x = self.block4(x)
109
110            emb = self.gap(x).flatten(1)
111            logits = self.fc(self.dropout(emb))
112
113            if return_embedding:
114                return logits, emb
115            return logits
116
117        # -----
118        # Novelty detection: convolutional autoencoder
119        # -----
120        class ConvAutoencoder(nn.Module):
121            """Compresses a spectrogram to a small latent code and rebuilds it.
122
123            Trained only on the ten known UrbanSound8K classes, it learns to
124            reconstruct
125            those well. A sound it has never encountered (a smashing window, a crying
126            baby) falls outside that learned manifold and reconstructs poorly, so the
127            per-clip reconstruction error works directly as a novelty score.
128            """
129
130            def __init__(self, latent_ch: int = 32):
131                super().__init__()
132
133                self.encoder = nn.Sequential(
134                    nn.Conv2d(1, 16, 3, stride=2, padding=1), # -> 32 x 87
135                    nn.BatchNorm2d(16),
136                    nn.ReLU(),
137                    nn.Conv2d(16, 32, 3, stride=2, padding=1), # -> 16 x 44
138                    nn.BatchNorm2d(32),
139                    nn.ReLU(),
140                    nn.Conv2d(32, latent_ch, 3, stride=2, padding=1), # -> 8 x 22
141                    nn.BatchNorm2d(latent_ch),
142                    nn.ReLU(),
143                )
144
145                self.decoder = nn.Sequential(

```

```

145         nn.ConvTranspose2d(latent_ch, 32, 3, stride=2, padding=1,
146                             output_padding=1),
147         nn.BatchNorm2d(32),
148         nn.ReLU(),
149         nn.ConvTranspose2d(32, 16, 3, stride=2, padding=1, output_padding
150                             =1),
151         nn.BatchNorm2d(16),
152         nn.ReLU(),
153         nn.ConvTranspose2d(16, 1, 3, stride=2, padding=1, output_padding=1)
154     )
155
156     def forward(self, x):
157         z = self.encoder(x)
158         out = self.decoder(z)
159         # Transposed convolutions can overshoot by a pixel; crop to match input
160         return out[..., : x.shape[-2], : x.shape[-1]]
161
162     def encode(self, x):
163         return self.encoder(x)
164
165     @torch.no_grad()
166     def reconstruction_error(self, x) -> torch.Tensor:
167         """Per-sample mean squared error the novelty score."""
168         recon = self(x)
169         return ((recon - x) ** 2).flatten(1).mean(dim=1)
170
171     # -----
172     # Helpers
173     # -----
174
175     def count_parameters(model: nn.Module) -> int:
176         return sum(p.numel() for p in model.parameters() if p.requires_grad)
177
178     def build_model(name: str, n_classes: int = 10, **kwargs) -> nn.Module:
179         name = name.lower()
180         if name == "mlp":
181             return MLPBaseline(n_classes=n_classes, **kwargs)
182         if name == "cnn":
183             return AudioCNN(n_classes=n_classes, **kwargs)
184         if name in ("autoencoder", "ae"):
185             return ConvAutoencoder(**kwargs)
186         raise ValueError(f"unknown model: {name}")

```

Listing 8: Network architectures: MLP, CNN, autoencoder (src/models.py)

D.4 src/train.py

Training loop and 10-fold cross-validation.

```

1  """Training engine: single-fold training plus the official 10-fold protocol."""
2  from __future__ import annotations
3
4  import json
5  import time
6  from dataclasses import dataclass, field, asdict
7
8  import numpy as np
9  import torch
10 import torch.nn as nn
11 from torch.utils.data import DataLoader

```



```

12
13 from . import config as C
14 from .data import SpectrogramDataset, compute_norm_stats, make_fold_split
15 from .models import build_model, count_parameters
16
17
18 def set_seed(seed: int = C.SEED) -> None:
19     import random
20
21     random.seed(seed)
22     np.random.seed(seed)
23     torch.manual_seed(seed)
24     torch.mps.manual_seed(seed) if torch.backends.mps.is_available() else None
25
26
27 @dataclass
28 class FoldResult:
29     fold: int
30     model: str
31     best_val_acc: float
32     test_acc: float
33     test_f1_macro: float
34     epochs_run: int
35     train_seconds: float
36     n_params: int
37     history: dict = field(default_factory=dict)
38
39
40 # -----
41 # Epoch loops
42 # -----
43
44 def _run_epoch(model, loader, criterion, optimizer, device, train: bool):
45     model.train() if train else model.eval()
46     total_loss, correct, total = 0.0, 0, 0
47
48     with torch.set_grad_enabled(train):
49         for xb, yb in loader:
50             xb, yb = xb.to(device), yb.to(device)
51
52             logits = model(xb)
53             loss = criterion(logits, yb)
54
55             if train:
56                 optimizer.zero_grad(set_to_none=True)
57                 loss.backward()
58                 optimizer.step()
59
60             total_loss += loss.item() * xb.size(0)
61             correct += (logits.argmax(1) == yb).sum().item()
62             total += xb.size(0)
63
64     return total_loss / total, correct / total
65
66
67 @torch.no_grad()
68 def predict(model, loader, device):
69     """Returns (y_true, y_pred, probabilities) for a whole loader."""
70     model.eval()
71     ys, ps, probs = [], [], []
72     for xb, yb in loader:
73         logits = model(xb.to(device))
74         p = torch.softmax(logits, dim=1)
75         ys.append(yb.numpy())

```

```

75         ps.append(logits.argmax(1).cpu().numpy())
76         probs.append(p.cpu().numpy())
77     return np.concatenate(ys), np.concatenate(ps), np.concatenate(probs)
78
79
80     # -----
81     # Single fold
82     # -----
83     def train_one_fold(
84         X,
85         meta,
86         test_fold: int,
87         model_name: str = "cnn",
88         epochs: int = C.EPOCHS,
89         batch_size: int = C.BATCH_SIZE,
90         lr: float = C.LEARNING_RATE,
91         augment: bool = True,
92         device: torch.device = C.DEVICE,
93         n_classes: int = 10,
94         verbose: bool = True,
95         save_path=None,
96     ) -> tuple[nn.Module, FoldResult]:
97         """Train one model on the folds outside 'test_fold', evaluate on it."""
98         from sklearn.metrics import f1_score
99
100         set_seed(C.SEED + test_fold)
101
102         (Xtr, ytr), (Xva, yva), (Xte, yte) = make_fold_split(X, meta, test_fold)
103
104         # Normalisation statistics come from the training split only.
105         mean, std = compute_norm_stats(Xtr)
106
107         ds_tr = SpectrogramDataset(Xtr, ytr, mean, std, augment=augment)
108         ds_va = SpectrogramDataset(Xva, yva, mean, std, augment=False)
109         ds_te = SpectrogramDataset(Xte, yte, mean, std, augment=False)
110
111         dl_tr = DataLoader(ds_tr, batch_size=batch_size, shuffle=True, drop_last=
            True)
112         dl_va = DataLoader(ds_va, batch_size=batch_size)
113         dl_te = DataLoader(ds_te, batch_size=batch_size)
114
115         model = build_model(model_name, n_classes=n_classes).to(device)
116         criterion = nn.CrossEntropyLoss(label_smoothing=0.05)
117         optimizer = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=C.
            WEIGHT_DECAY)
118         scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=
            epochs)
119
120         history = {"train_loss": [], "train_acc": [], "val_loss": [], "val_acc":
            []}
121         best_val, best_state, patience, t0 = 0.0, None, 0, time.time()
122         epochs_run = 0
123
124         for ep in range(1, epochs + 1):
125             tr_loss, tr_acc = _run_epoch(model, dl_tr, criterion, optimizer, device
                , True)
126             va_loss, va_acc = _run_epoch(model, dl_va, criterion, optimizer, device
                , False)
127             scheduler.step()
128             epochs_run = ep
129
130             history["train_loss"].append(tr_loss)
131             history["train_acc"].append(tr_acc)

```

```

132     history["val_loss"].append(va_loss)
133     history["val_acc"].append(va_acc)
134
135     if va_acc > best_val:
136         best_val = va_acc
137         best_state = {k: v.detach().cpu().clone() for k, v in model.
138                       state_dict().items()}
139         patience = 0
140     else:
141         patience += 1
142
143     if verbose and (ep % 5 == 0 or ep == 1):
144         print(
145             f"  fold{test_fold} {model_name} ep{ep:3d}/{epochs} "
146             f"train {tr_loss:.3f}/{tr_acc:.3f}  val {va_loss:.3f}/{va_acc
147               :.3f}"
148         )
149
150     if patience >= C.EARLY_STOP_PATIENCE:
151         if verbose:
152             print(f"  fold{test_fold} early stop at epoch {ep}")
153         break
154
155     if best_state is not None:
156         model.load_state_dict(best_state)
157
158     y_true, y_pred, _ = predict(model, dl_te, device)
159     test_acc = float((y_true == y_pred).mean())
160     test_f1 = float(f1_score(y_true, y_pred, average="macro"))
161
162     result = FoldResult(
163         fold=test_fold,
164         model=model_name,
165         best_val_acc=best_val,
166         test_acc=test_acc,
167         test_f1_macro=test_f1,
168         epochs_run=epochs_run,
169         train_seconds=time.time() - t0,
170         n_params=count_parameters(model),
171         history=history,
172     )
173
174     if save_path:
175         torch.save(
176             {"state_dict": model.state_dict(), "mean": mean, "std": std,
177              "model_name": model_name, "n_classes": n_classes},
178             save_path,
179         )
180
181     if verbose:
182         print(
183             f"  fold{test_fold} {model_name} -> test acc {test_acc:.4f} "
184             f"f1 {test_f1:.4f} ({result.train_seconds:.0f}s)"
185         )
186
187     return model, result
188
189 # -----
190 # Full cross-validation
191 # -----
192 def cross_validate(
193     X,

```

```

193     meta,
194     model_name: str = "cnn",
195     folds=None,
196     epochs: int = C.EPOCHS,
197     augment: bool = True,
198     tag: str | None = None,
199     **kwargs,
200 ):
201     """Run the official 10-fold protocol and report mean +/- std.
202
203     Two conveniences for long unattended runs, both controlled by environment
204     variables so they never affect the results:
205
206     * ‘COOLDOWN_SEC’ pauses between folds to let the GPU shed heat. Folds are
207     independent, so this changes timing only, never the numbers.
208     * ‘RESUME=1’ skips a configuration whose cached JSON already covers the
209     requested folds, so a restarted run does not redo finished work. Results
210     are deterministic, so a skipped config is identical to a rerun.
211     """
212     import os
213
214     folds = folds or list(range(1, C.N_FOLDS + 1))
215     cooldown = float(os.environ.get("COOLDOWN_SEC", "0"))
216
217     name = tag or f"{model_name}_aug{int(augment)}"
218     out = C.LOGS_DIR / f"cv_{name}.json"
219
220     if os.environ.get("RESUME") == "1" and out.exists():
221         cached = json.loads(out.read_text())
222         if set(cached.get("folds", [])) >= set(folds):
223             print(f"[resume] {name} already complete "
224                   f"({cached['acc_mean']*100:.2f}%) -- skipping")
225             return cached
226
227     results: list[FoldResult] = []
228
229     print(f"\n{'='*70}\n{model_name.upper()} | folds={folds} augment={
230           augment}")
231     f"{f'   cooldown={cooldown:.0f}s' if cooldown else ''}\n{'='*70}"
232
233     for i, f in enumerate(folds):
234         _, res = train_one_fold(
235             X, meta, test_fold=f, model_name=model_name,
236             epochs=epochs, augment=augment, **kwargs
237         )
238         results.append(res)
239
240         if cooldown and i < len(folds) - 1:
241             print(f"   [cooldown] pausing {cooldown:.0f}s to let the GPU cool "
242                   f"(fold {i+1}/{len(folds)} done)")
243             time.sleep(cooldown)
244
245     accs = np.array([r.test_acc for r in results])
246     f1s = np.array([r.test_f1_macro for r in results])
247
248     summary = {
249         "model": model_name,
250         "augment": augment,
251         "folds": folds,
252         "acc_mean": float(accs.mean()), "acc_std": float(accs.std()),
253         "f1_mean": float(f1s.mean()), "f1_std": float(f1s.std()),
254         "n_params": results[0].n_params,
255         "total_seconds": float(sum(r.train_seconds for r in results)),

```

```

255     "per_fold": [asdict(r) for r in results],
256 }
257
258 print(
259     f"\n{model_name.upper()} SUMMARY  acc {accs.mean():.4f} +/- {accs.std()
260         :.4f} | "
261     f"macro-F1 {f1s.mean():.4f} +/- {f1s.std():.4f} | params {results[0].
262         n_params:}"
263 )
264
265 name = tag or f"{model_name}_aug{int(augment)}"
266 out = C.LOGS_DIR / f"cv_{name}.json"
267 out.write_text(json.dumps(summary, indent=2))
268 print(f"saved -> {out}")
269
270 return summary

```

Listing 9: Training loop and 10-fold cross-validation (src/train.py)

D.5 src/evaluate.py

Evaluation and embedding extraction.

```

1  """Evaluation: aggregated cross-validation metrics and embedding extraction."""
2  from __future__ import annotations
3
4  import numpy as np
5  import torch
6  from torch.utils.data import DataLoader
7
8  from . import config as C
9  from .data import SpectrogramDataset, compute_norm_stats, make_fold_split
10 from .train import predict, train_one_fold
11
12
13 def evaluate_cv_predictions(X, meta, model_name="cnn", folds=None,
14                             epochs=C.EPOCHS, augment=True, **kwargs):
15     """Train per fold and pool the held-out predictions.
16
17     Because every clip is in exactly one test fold, pooling gives one honest
18     prediction for the entire dataset the right basis for an aggregate
19     confusion matrix and per-class scores.
20     """
21     import os
22     import time
23
24     folds = folds or list(range(1, C.N_FOLDS + 1))
25     cooldown = float(os.environ.get("COOLDOWN_SEC", "0"))
26     all_true, all_pred, all_prob, results = [], [], [], []
27
28     for i, f in enumerate(folds):
29         model, res = train_one_fold(
30             X, meta, test_fold=f, model_name=model_name,
31             epochs=epochs, augment=augment, **kwargs
32         )
33         results.append(res)
34
35         (Xtr, ytr), _, (Xte, yte) = make_fold_split(X, meta, f)
36         mean, std = compute_norm_stats(Xtr)
37         dl = DataLoader(SpectrogramDataset(Xte, yte, mean, std), batch_size=C.
38             BATCH_SIZE)
39         yt, yp, pr = predict(model, dl, C.DEVICE)

```

```

40     all_true.append(yt); all_pred.append(yp); all_prob.append(pr)
41
42     if cooldown and i < len(folds) - 1:
43         print(f" [cooldown] pausing {cooldown:.0f}s (eval fold {i+1}/{len(
44             folds)} done)")
45         time.sleep(cooldown)
46
47     return (np.concatenate(all_true), np.concatenate(all_pred),
48             np.concatenate(all_prob), results)
49
50 def full_metrics(y_true, y_pred, class_names) -> dict:
51     from sklearn.metrics import (accuracy_score, classification_report,
52                                 confusion_matrix, f1_score,
53                                 precision_score, recall_score)
54
55     return {
56         "accuracy": float(accuracy_score(y_true, y_pred)),
57         "precision_macro": float(precision_score(y_true, y_pred, average="macro",
58             zero_division=0)),
59         "recall_macro": float(recall_score(y_true, y_pred, average="macro",
60             zero_division=0)),
61         "f1_macro": float(f1_score(y_true, y_pred, average="macro",
62             zero_division=0)),
63         "f1_weighted": float(f1_score(y_true, y_pred, average="weighted",
64             zero_division=0)),
65         "per_class_f1": {
66             c: float(v) for c, v in zip(
67                 class_names, f1_score(y_true, y_pred, average=None,
68                     zero_division=0)
69             )
70         },
71         "confusion_matrix": confusion_matrix(y_true, y_pred).tolist(),
72         "report": classification_report(
73             y_true, y_pred, target_names=class_names, zero_division=0, digits=3
74         ),
75     }
76
77 @torch.no_grad()
78 def extract_embeddings(model, X, y, mean, std, device=C.DEVICE, batch_size=C.
79     BATCH_SIZE):
80     """Pull the CNN's 256-d penultimate features for clustering / t-SNE."""
81     model.eval()
82     ds = SpectrogramDataset(X, y, mean, std, augment=False)
83     dl = DataLoader(ds, batch_size=batch_size)
84
85     embs, labels = [], []
86     for xb, yb in dl:
87         _, emb = model(xb.to(device), return_embedding=True)
88         embs.append(emb.cpu().numpy())
89         labels.append(yb.numpy())
90
91     return np.concatenate(embs), np.concatenate(labels)

```

Listing 10: Evaluation and embedding extraction (src/evaluate.py)

D.6 src/anomaly.py

Autoencoder novelty detection.

```

1 """Novel-sound detection with a convolutional autoencoder.
2

```

```

3  The classifier is closed-set: shown a smashing window it must still answer with
4  one of its ten labels, usually confidently and wrongly. For an assistive system
5  that is the dangerous failure mode.
6
7  The fix here is an autoencoder trained *only* on the ten known classes. It
8  learns
9  to rebuild those, so an unfamiliar sound reconstructs badly and its
10 reconstruction
11 error becomes a usable "I have not heard this before" score.
12 """
13 from __future__ import annotations
14
15 import time
16
17 import numpy as np
18 import torch
19 import torch.nn as nn
20 from torch.utils.data import DataLoader
21
22 from . import config as C
23 from .data import SpectrogramDataset
24 from .models import ConvAutoencoder
25 from .train import set_seed
26
27 def train_autoencoder(X_train, mean, std, epochs=30, batch_size=C.BATCH_SIZE,
28                       lr=1e-3, device=C.DEVICE, verbose=True, X_val=None):
29     """Fit the autoencoder on known-class spectrograms only."""
30     set_seed(C.SEED)
31
32     dummy = np.zeros(len(X_train), dtype=np.int64)
33     dl = DataLoader(SpectrogramDataset(X_train, dummy, mean, std, augment=False
34                                      ),
35                    batch_size=batch_size, shuffle=True, drop_last=True)
36
37     dl_val = None
38     if X_val is not None:
39         dl_val = DataLoader(
40             SpectrogramDataset(X_val, np.zeros(len(X_val), dtype=np.int64),
41                                 mean, std),
42             batch_size=batch_size)
43
44     model = ConvAutoencoder().to(device)
45     criterion = nn.MSELoss()
46     optimizer = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=1e-5)
47     scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=
48                                                             epochs)
49
50     history = {"train_loss": [], "val_loss": []}
51     t0 = time.time()
52
53     for ep in range(1, epochs + 1):
54         model.train()
55         total, n = 0.0, 0
56         for xb, _ in dl:
57             xb = xb.to(device)
58             loss = criterion(model(xb), xb)
59             optimizer.zero_grad(set_to_none=True)
60             loss.backward()
61             optimizer.step()
62             total += loss.item() * xb.size(0); n += xb.size(0)
63         scheduler.step()
64         history["train_loss"].append(total / n)

```

```

61
62     if dl_val is not None:
63         model.eval()
64         vt, vn = 0.0, 0
65         with torch.no_grad():
66             for xb, _ in dl_val:
67                 xb = xb.to(device)
68                 vt += criterion(model(xb), xb).item() * xb.size(0); vn +=
69                     xb.size(0)
70             history["val_loss"].append(vt / vn)
71
72     if verbose and (ep % 5 == 0 or ep == 1):
73         msg = f" AE ep{ep:3d}/{epochs} train {history['train_loss'][-1]:.5f}"
74         if history["val_loss"]:
75             msg += f" val {history['val_loss'][-1]:.5f}"
76         print(msg)
77
78     if verbose:
79         print(f" AE trained in {time.time()-t0:.0f}s")
80     return model, history
81
82 @torch.no_grad()
83 def reconstruction_scores(model, X, mean, std, device=C.DEVICE,
84                           batch_size=C.BATCH_SIZE) -> np.ndarray:
85     """Per-clip reconstruction error higher means less familiar."""
86     model.eval()
87     dl = DataLoader(SpectrogramDataset(X, np.zeros(len(X), dtype=np.int64),
88                                         mean, std),
89                     batch_size=batch_size)
89     out = []
90     for xb, _ in dl:
91         out.append(model.reconstruction_error(xb.to(device)).cpu().numpy())
92     return np.concatenate(out)
93
94
95 def evaluate_novelty(known_scores: np.ndarray, novel_scores: np.ndarray,
96                     percentile: float = 95.0) -> dict:
97     """Score the detector as a binary problem: novel (1) vs known (0).
98
99     The threshold is set at a percentile of the KNOWN scores, so it can be
100    calibrated without ever seeing a novel example which is the situation a
101    deployed system is actually in.
102    """
103     from sklearn.metrics import (average_precision_score, roc_auc_score,
104                                   roc_curve)
105
106     y = np.concatenate([np.zeros(len(known_scores)), np.ones(len(novel_scores))
107                          ])
108     s = np.concatenate([known_scores, novel_scores])
109
110     auc = float(roc_auc_score(y, s))
111     ap = float(average_precision_score(y, s))
112     fpr, tpr, _ = roc_curve(y, s)
113
114     thr = float(np.percentile(known_scores, percentile))
115     tp = int((novel_scores > thr).sum())
116     fn = int((novel_scores <= thr).sum())
117     fp = int((known_scores > thr).sum())
118     tn = int((known_scores <= thr).sum())
119
120     precision = tp / (tp + fp) if (tp + fp) else 0.0

```



```

119 recall = tp / (tp + fn) if (tp + fn) else 0.0
120 f1 = 2 * precision * recall / (precision + recall) if (precision + recall)
    else 0.0
121
122 return {
123     "roc_auc": auc,
124     "average_precision": ap,
125     "threshold": thr,
126     "threshold_percentile": percentile,
127     "precision": precision,
128     "recall": recall,
129     "f1": f1,
130     "confusion": {"tp": tp, "fp": fp, "tn": tn, "fn": fn},
131     "fpr": fpr.tolist(),
132     "tpr": tpr.tolist(),
133     "known_mean": float(known_scores.mean()),
134     "novel_mean": float(novel_scores.mean()),
135 }
136
137
138 def softmax_confidence_baseline(probs_known: np.ndarray,
139                                probs_novel: np.ndarray) -> dict:
140     """Baseline: use 1 - max(softmax) as the novelty score.
141
142     Included so the report can show whether the autoencoder actually beats the
143     obvious "the classifier is unsure" heuristic, rather than assuming it does.
144     """
145     from sklearn.metrics import roc_auc_score
146
147     known = 1.0 - probs_known.max(axis=1)
148     novel = 1.0 - probs_novel.max(axis=1)
149     y = np.concatenate([np.zeros(len(known)), np.ones(len(novel))])
150     s = np.concatenate([known, novel])
151     return {
152         "roc_auc": float(roc_auc_score(y, s)),
153         "known_mean_conf": float(probs_known.max(axis=1).mean()),
154         "novel_mean_conf": float(probs_novel.max(axis=1).mean()),
155     }

```

Listing 11: Autoencoder novelty detection (src/anomaly.py)

D.7 src/cluster.py

k-means and t-SNE clustering.

```

1 """Unsupervised analysis of the CNN's learned representation.
2
3 The classifier is trained with labels, but the *embedding space* it builds can
4 be
5 examined without them. If k-means on those embeddings recovers the sound
6 classes,
7 the network has learned a genuinely structured representation rather than a
8 bare
9 decision rule and the classes it confuses show up as overlapping clusters.
10 """
11 from __future__ import annotations
12
13 import numpy as np
14
15 from . import config as C
16
17 def run_tsne(embeddings: np.ndarray, n_components: int = 2,

```

```

16         perplexity: float = 30.0, max_samples: int = 3000,
17         seed: int = C.SEED):
18     """Project embeddings to 2-D for visualisation.
19
20 t-SNE is  $O(n^2)$ -ish, so large inputs are subsampled. Returns (points, index
21 )
22 where 'index' selects the rows that were kept.
23     """
24     from sklearn.manifold import TSNE
25
26     rng = np.random.default_rng(seed)
27     idx = (rng.choice(len(embeddings), max_samples, replace=False)
28            if len(embeddings) > max_samples else np.arange(len(embeddings)))
29
30     tsne = TSNE(n_components=n_components, perplexity=perplexity,
31                init="pca", learning_rate="auto", random_state=seed)
32     return tsne.fit_transform(embeddings[idx]), idx
33
34 def run_kmeans(embeddings: np.ndarray, n_clusters: int = 10, seed: int = C.SEED
35 ):
36     from sklearn.cluster import KMeans
37
38     km = KMeans(n_clusters=n_clusters, random_state=seed, n_init=10)
39     return km.fit_predict(embeddings), km
40
41 def clustering_metrics(embeddings: np.ndarray, cluster_labels: np.ndarray,
42                       true_labels: np.ndarray) -> dict:
43     """Compare discovered clusters against the true classes.
44
45 ARI/NMI are label-permutation invariant, which matters because cluster 3
46 has
47 no reason to correspond to class 3. Silhouette judges cluster separation
48 without using labels at all.
49     """
50     from sklearn.metrics import (adjusted_rand_score,
51                                  normalized_mutual_info_score, silhouette_score)
52
53     return {
54         "adjusted_rand_index": float(adjusted_rand_score(true_labels,
55                                                         cluster_labels)),
56         "normalized_mutual_info": float(normalized_mutual_info_score(
57             true_labels, cluster_labels)),
58         "silhouette": float(silhouette_score(embeddings, cluster_labels)),
59         "n_clusters": int(len(np.unique(cluster_labels))),
60     }
61
62 def cluster_purity(cluster_labels: np.ndarray, true_labels: np.ndarray,
63                   class_names: list[str]) -> dict:
64     """For each cluster, the dominant true class and how pure the cluster is.
65     """
66     out = {}
67     for c in np.unique(cluster_labels):
68         m = cluster_labels == c
69         vals, counts = np.unique(true_labels[m], return_counts=True)
70         top = vals[counts.argmax()]
71         out[int(c)] = {
72             "dominant_class": class_names[int(top)],
73             "purity": float(counts.max() / counts.sum()),
74             "size": int(m.sum()),

```

```

72     }
73     return out

```

Listing 12: k-means and t-SNE clustering (src/cluster.py)

D.8 src/viz.py

Figure generation.

```

1  """Figure generation for the report.
2
3  All figures share one visual system: a validated categorical palette, a single
4  blue sequential ramp for magnitude, recessive grid/axis ink, and direct labels
5  so identity never rests on colour alone.
6  """
7  from __future__ import annotations
8
9  import matplotlib as mpl
10 import matplotlib.pyplot as plt
11 import numpy as np
12
13 from . import config as C
14
15 # -----
16 # Design tokens (validated: all-pairs CVD dE 9.2, normal-vision 24.0, light)
17 # -----
18 SERIES = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4",
19           "#008300", "#4a3aa7", "#e34948", "#6da7ec", "#c98500"]
20
21 SEQ_BLUE = ["#cde2fb", "#b7d3f6", "#9ec5f4", "#86b6ef", "#6da7ec",
22            "#5598e7", "#3987e5", "#2a78d6", "#256abf", "#1c5cab",
23            "#184f95", "#104281", "#0d366b"]
24
25 SURFACE = "#fcfcfb"
26 INK = "#0b0b0b"
27 INK_2 = "#52514e"
28 MUTED = "#898781"
29 GRID = "#e1e0d9"
30 AXIS = "#c3c2b7"
31
32 STATUS = {"good": "#0ca30c", "warning": "#fab219", "critical": "#d03b3b"}
33
34
35 def use_report_style() -> None:
36     """Apply the shared style. Call once before generating figures."""
37     mpl.rcParams.update({
38         "figure.facecolor": SURFACE,
39         "axes.facecolor": SURFACE,
40         "savefig.facecolor": SURFACE,
41         "font.family": "sans-serif",
42         "font.sans-serif": ["Helvetica Neue", "Helvetica", "Arial", "DejaVu
43                             Sans"],
44         "font.size": 10,
45         "axes.titlesize": 12,
46         "axes.titleweight": "bold",
47         "axes.labelsize": 10,
48         "axes.labelcolor": INK_2,
49         "axes.edgecolor": AXIS,
50         "axes.linewidth": 0.8,
51         "axes.grid": True,
52         "axes.axisbelow": True,
53         "grid.color": GRID,
54         "grid.linewidth": 0.7,

```

```

54         "xtick.color": MUTED,
55         "ytick.color": MUTED,
56         "xtick.labelsize": 9,
57         "ytick.labelsize": 9,
58         "text.color": INK,
59         "legend.frameon": False,
60         "legend.fontsize": 9,
61         "figure.dpi": 130,
62         "savefig.dpi": 300,           # print-resolution figures for the report
63         "savefig.bbox": "tight",
64     })
65
66
67 def _despine(ax, keep=("left", "bottom")):
68     for side in ("top", "right", "left", "bottom"):
69         ax.spines[side].set_visible(side in keep)
70
71
72 def save(fig, name: str) -> str:
73     """Save a figure to results/figures as PNG (report) and PDF (vector)."""
74     path = C.FIGURES_DIR / f"{name}.png"
75     fig.savefig(path)
76     fig.savefig(C.FIGURES_DIR / f"{name}.pdf")
77     plt.close(fig)
78     print(f" figure -> {path}")
79     return str(path)
80
81
82 # -----
83 # Dataset overview
84 # -----
85 def plot_class_distribution(counts: dict[str, int], name="
fig_class_distribution",
86                             title="UrbanSound8K clips per class"):
87     labels = list(counts.keys())
88     values = [counts[k] for k in labels]
89     order = np.argsort(values)[::-1]
90     labels = [labels[i].replace("_", " ") for i in order]
91     values = [values[i] for i in order]
92
93     fig, ax = plt.subplots(figsize=(7.5, 4.2))
94     bars = ax.barh(labels[::-1], values[::-1], color=SERIES[0], height=0.62)
95     for b, v in zip(bars, values[::-1]):
96         ax.text(v + max(values) * 0.012, b.get_y() + b.get_height() / 2,
97                f"{v:,}", va="center", ha="left", fontsize=9, color=INK_2)
98
99     ax.set_xlabel("Number of clips")
100    ax.set_title(title, loc="left", pad=12)
101    ax.set_xlim(0, max(values) * 1.12)
102    ax.grid(axis="y", visible=False)
103    _despine(ax)
104    return save(fig, name)
105
106
107 def plot_spectrogram_grid(specs, titles, name="fig_spectrogram_examples",
108                             subtitle="Log-mel spectrograms: each sound has a
distinct signature",
109                             ncols=5):
110     n = len(specs)
111     nrows = int(np.ceil(n / ncols))
112     fig, axes = plt.subplots(nrows, ncols, figsize=(2.6 * ncols, 2.3 * nrows))
113     axes = np.atleast_1d(axes).ravel()
114

```

```

115     for ax, spec, t in zip(axes, specs, titles):
116         ax.imshow(spec, origin="lower", aspect="auto", cmap="magma",
117                   extent=[0, C.CLIP_SECONDS, 0, C.N_MELS])
118         ax.set_title(t.replace("_", " "), fontsize=9.5, pad=5)
119         ax.set_xticks([0, 2, 4]); ax.set_yticks([])
120         ax.grid(False)
121         ax.tick_params(labelsize=8)
122     for ax in axes[n:]:
123         ax.axis("off")
124
125     fig.suptitle(suptitle, fontsize=12, fontweight="bold", x=0.02, ha="left", y
126                 =1.0)
127     fig.supxlabel("Time (s)", fontsize=9, color=INK_2)
128     fig.supylabel("Mel frequency band", fontsize=9, color=INK_2)
129     fig.tight_layout()
130     return save(fig, name)
131
132 # -----
133 # Training
134 # -----
135 def plot_training_curves(histories: dict[str, dict], name="fig_training_curves"
136 ):
137     """histories: {model_label: history_dict}"""
138     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 3.8))
139
140     for i, (label, h) in enumerate(histories.items()):
141         c = SERIES[i]
142         ep = range(1, len(h["train_loss"]) + 1)
143         ax1.plot(ep, h["train_loss"], color=c, lw=2, ls="--", alpha=0.55)
144         ax1.plot(ep, h["val_loss"], color=c, lw=2, label=label)
145         ax2.plot(ep, [a * 100 for a in h["train_acc"]], color=c, lw=2, ls="--",
146                   alpha=0.55)
147         ax2.plot(ep, [a * 100 for a in h["val_acc"]], color=c, lw=2, label=
148                   label)
149         # direct label at the end of the validation curve
150         ax2.annotate(label, (len(h["val_acc"]), h["val_acc"][-1] * 100),
151                     xytext=(4, 0), textcoords="offset points",
152                     color=c, fontsize=9, fontweight="bold", va="center")
153
154     ax1.set_xlabel("Epoch"); ax1.set_ylabel("Cross-entropy loss")
155     ax1.set_title("Loss (dashed = train, solid = validation)", loc="left", pad
156                 =10)
157     ax2.set_xlabel("Epoch"); ax2.set_ylabel("Accuracy (%)")
158     ax2.set_title("Accuracy (dashed = train, solid = validation)", loc="left",
159                 pad=10)
160     ax1.legend(loc="upper right")
161     for ax in (ax1, ax2):
162         _despine(ax)
163     fig.tight_layout()
164     return save(fig, name)
165
166 def plot_confusion_matrix(cm, classes, name="fig_confusion_matrix",
167                          title="CNN confusion matrix (10-fold aggregate)",
168                          normalize=True):
169     from matplotlib.colors import LinearSegmentedColormap
170
171     cmap = LinearSegmentedColormap.from_list("seqblue", SEQ_BLUE)
172     m = cm.astype(float)
173     if normalize:
174         m = m / np.clip(m.sum(axis=1, keepdims=True), 1, None) * 100

```

```

171 fig, ax = plt.subplots(figsize=(7.6, 6.4))
172 im = ax.imshow(m, cmap=cmap, vmin=0, vmax=100 if normalize else m.max())
173
174 labels = [c.replace("_", " ") for c in classes]
175 ax.set_xticks(range(len(classes)), labels, rotation=45, ha="right")
176 ax.set_yticks(range(len(classes)), labels)
177 ax.set_xlabel("Predicted"); ax.set_ylabel("Actual")
178 ax.set_title(title, loc="left", pad=12)
179 ax.grid(False)
180
181 # value labels identity never rests on colour alone
182 for i in range(m.shape[0]):
183     for j in range(m.shape[1]):
184         v = m[i, j]
185         if v >= 0.5:
186             ax.text(j, i, f"{v:.0f}", ha="center", va="center", fontsize
187                     =8.5,
188                     color="#ffffff" if v > 55 else INK_2,
189                     fontweight="bold" if i == j else "normal")
190
191 cb = fig.colorbar(im, ax=ax, fraction=0.045, pad=0.03)
192 cb.set_label("% of actual class" if normalize else "clips", color=INK_2,
193             fontsize=9)
194 cb.outline.set_visible(False)
195 fig.tight_layout()
196 return save(fig, name)
197
198 def plot_model_comparison(results: dict[str, dict], name="fig_model_comparison"
199 ,
200                             n_folds: int | None = None):
201     """results: {label: {'acc_mean', 'acc_std', 'f1_mean', 'f1_std', 'n_params'}}"""
202     labels = list(results.keys())
203     x = np.arange(len(labels))
204     w = 0.36
205
206     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10.5, 4.3),
207                                   gridspec_kw={"width_ratios": [1.55, 1]})
208
209     acc = [results[l]["acc_mean"] * 100 for l in labels]
210     accs = [results[l]["acc_std"] * 100 for l in labels]
211     f1 = [results[l]["f1_mean"] * 100 for l in labels]
212     f1s = [results[l]["f1_std"] * 100 for l in labels]
213
214     b1 = ax1.bar(x - w / 2, acc, w, yerr=accs, capsize=3, color=SERIES[0],
215                 label="Accuracy")
216     b2 = ax1.bar(x + w / 2, f1, w, yerr=f1s, capsize=3, color=SERIES[1], label=
217               "Macro F1")
218
219     headroom = max(v + e for v, e in zip(acc + f1, accs + f1s))
220     # Labels clear the error-bar cap, not just the bar top.
221     for bars, vals, errs in ((b1, acc, accs), (b2, f1, f1s)):
222         for b, v, e in zip(bars, vals, errs):
223             ax1.text(b.get_x() + b.get_width() / 2, v + e + headroom * 0.035, f
224                     "{v:.1f}",
225                     ha="center", fontsize=9, color=INK_2, fontweight="bold")
226
227     fold_note = f"{n_folds}-fold " if n_folds else ""
228     ax1.set_xticks(x, labels)
229     ax1.set_ylabel("Score (%)")
230     ax1.set_ylim(0, headroom * 1.30)
231     ax1.set_title(f"Performance ({fold_note}mean ± sd)", loc="left", pad=10)

```

```

227     # Legend on the title line but right-aligned: clear of both the
228     # left-aligned title and every bar.
229     ax1.legend(loc="lower right", bbox_to_anchor=(1.0, 1.005), ncols=2,
230               frameon=False, borderaxespad=0)
231     ax1.grid(axis="x", visible=False)
232     _despine(ax1)
233
234     params = [results[l]["n_params"] / 1e6 for l in labels]
235     b3 = ax2.bar(x, params, 0.5, color=SERIES[2])
236     for b, v in zip(b3, params):
237         ax2.text(b.get_x() + b.get_width() / 2, v + max(params) * 0.03,
238                 f"{v:.2f}M", ha="center", fontsize=9, color=INK_2, fontweight=
239                     "bold")
240     ax2.set_xticks(x, labels)
241     ax2.set_ylabel("Trainable parameters (millions)")
242     ax2.set_ylim(0, max(params) * 1.22)
243     ax2.set_title("Model size", loc="left", pad=10)
244     ax2.grid(axis="x", visible=False)
245     _despine(ax2)
246
247     fig.tight_layout()
248     return save(fig, name)
249
250 def plot_per_class_f1(per_class: dict[str, float], name="fig_per_class_f1"):
251     items = sorted(per_class.items(), key=lambda kv: kv[1])
252     labels = [k.replace("_", " ") for k, _ in items]
253     vals = [v * 100 for _, v in items]
254     colors = [STATUS["critical"] if v < 70 else (STATUS["warning"] if v < 85
255         else SERIES[0])
256               for v in vals]
257
258     fig, ax = plt.subplots(figsize=(7.5, 4.2))
259     bars = ax.barh(labels, vals, color=colors, height=0.62)
260     for b, v in zip(bars, vals):
261         ax.text(v + 1, b.get_y() + b.get_height() / 2, f"{v:.1f}",
262                 va="center", fontsize=9, color=INK_2)
263     ax.set_xlabel("F1 score (%)")
264     ax.set_xlim(0, 105)
265     ax.set_title("Per-class F1 which sounds the CNN finds hard", loc="left",
266                 pad=12)
267     ax.grid(axis="y", visible=False)
268     _despine(ax)
269     return save(fig, name)
270
271 # -----
272 # Embeddings / clustering
273 # -----
274 def plot_tsne(emb2d, labels, class_names, name="fig_tsne_embeddings",
275              title="What the CNN learned: t-SNE of 256-d embeddings"):
276     """Colour + a direct centroid label per class, so identity is never
277     carried by colour alone (10 classes exceeds a safe colour-only cap)."""
278     fig, ax = plt.subplots(figsize=(8.2, 6.8))
279
280     for i, cname in enumerate(class_names):
281         m = labels == i
282         if not m.any():
283             continue
284         ax.scatter(emb2d[m, 0], emb2d[m, 1], s=9, alpha=0.55,
285                   color=SERIES[i % len(SERIES)], linewidths=0)
286         cx, cy = emb2d[m, 0].mean(), emb2d[m, 1].mean()
287         ax.annotate(cname.replace("_", " "), (cx, cy), fontsize=9.5, fontweight

```

```

        = "bold",
        ha="center", va="center", color=INK,
        bbox=dict(boxstyle="round,pad=0.28", fc=SURFACE,
                    ec=SERIES[i % len(SERIES)], lw=1.4, alpha=0.92))

287
288
289
290
291 ax.set_xticks([]); ax.set_yticks([])
292 ax.set_title(title, loc="left", pad=12)
293 ax.grid(False)
294 _despine(ax, keep=())
295 fig.tight_layout()
296 return save(fig, name)
297
298
299 # -----
300 # Anomaly / novelty detection
301 # -----
302 def plot_anomaly_scores(known, novel, threshold=None, name="fig_anomaly_scores"
303 ):
304     fig, ax = plt.subplots(figsize=(8, 4.2))
305     bins = np.linspace(0, np.percentile(np.concatenate([known, novel]), 99),
306                          60)
307
308     ax.hist(known, bins=bins, color=SERIES[0], alpha=0.75, label="Known sounds
309              (UrbanSound8K)")
310     ax.hist(novel, bins=bins, color=SERIES[1], alpha=0.75, label="Novel sounds
311              (ESC-50, unseen)")
312
313     if threshold is not None:
314         ax.axvline(threshold, color=STATUS["critical"], lw=2, ls="--")
315         ax.annotate(f"threshold {threshold:.3f}", (threshold, ax.get_ylim()[1]
316             * 0.92),
317                    xytext=(7, 0), textcoords="offset points",
318                    color=STATUS["critical"], fontsize=9, fontweight="bold")
319
320     ax.set_xlabel("Autoencoder reconstruction error")
321     ax.set_ylabel("Number of clips")
322     ax.set_title("Novelty detection: unseen sounds reconstruct worse", loc="
323         left", pad=12)
324     ax.legend(loc="upper right")
325     ax.grid(axis="x", visible=False)
326     _despine(ax)
327     return save(fig, name)
328
329
330
331
332 def plot_roc(fpr, tpr, auc, name="fig_anomaly_roc"):
333     fig, ax = plt.subplots(figsize=(5.2, 4.8))
334     ax.plot([0, 1], [0, 1], color=AXIS, lw=1.4, ls="--")
335     ax.plot(fpr, tpr, color=SERIES[0], lw=2.4)
336     ax.annotate(f"AUC = {auc:.3f}", (0.55, 0.22), fontsize=12, fontweight="bold
337         ", color=SERIES[0])
338     ax.set_xlabel("False positive rate"); ax.set_ylabel("True positive rate")
339     ax.set_title("Novel-sound detection ROC", loc="left", pad=12)
340     ax.set_xlim(0, 1); ax.set_ylim(0, 1.02)
341     _despine(ax)
342     fig.tight_layout()
343     return save(fig, name)
344
345
346
347 def plot_reconstructions(originals, recons, titles, name="
348     fig_ae_reconstructions"):
349     n = len(originals)
350     fig, axes = plt.subplots(2, n, figsize=(2.5 * n, 4.6))
351     axes = np.atleast_2d(axes)

```



```

341     for j in range(n):
342         for row, (data, tag) in enumerate(((originals[j], "input"), (recons[j],
343             "rebuilt"))):
344             ax = axes[row, j]
345             ax.imshow(data, origin="lower", aspect="auto", cmap="magma")
346             ax.set_xticks([]); ax.set_yticks([]); ax.grid(False)
347             if row == 0:
348                 ax.set_title(titles[j], fontsize=9.5, pad=5)
349             if j == 0:
350                 ax.set_ylabel(tag, fontsize=9.5, color=INK_2)
351             fig.suptitle("Autoencoder reconstructions known sounds rebuild cleanly,
352                 novel ones do not",
353                 fontsize=12, fontweight="bold", x=0.02, ha="left")
354     fig.tight_layout()
355     return save(fig, name)

```

Listing 13: Figure generation (src/viz.py)

D.9 run_experiments.py

Staged experiment runner.

```

1  #!/usr/bin/env python
2  """End-to-end experiment runner.
3
4  Stages (select with --stage, default: all)
5
6  prep      extract and cache mel-spectrograms for both datasets
7  explore   dataset figures (class balance, example spectrograms)
8  compare   MLP vs CNN, and CNN with/without augmentation
9  final     full 10-fold CNN run -> confusion matrix, per-class F1, saved model
10 cluster   k-means + t-SNE over the CNN's learned embeddings
11 anomaly   autoencoder novelty detection vs unseen ESC-50 sounds
12
13 Examples
14 python run_experiments.py --stage prep
15 python run_experiments.py --stage compare --folds 1 2 3 --epochs 25
16 python run_experiments.py                               # everything, full protocol
17 """
18 from __future__ import annotations
19
20 import argparse
21 import json
22 import time
23
24 import numpy as np
25 import torch
26
27 from src import config as C
28 from src import viz
29 from src.data import (load_esc50_metadata, load_us8k_metadata,
30                      compute_norm_stats, make_fold_split, precompute_features)
31
32
33 def _save(name: str, obj) -> None:
34     p = C.LOGS_DIR / f"{name}.json"
35     p.write_text(json.dumps(obj, indent=2, default=str))
36     print(f"    saved -> {p}")
37
38
39 def banner(msg: str) -> None:
40     print(f"\n{'='*74}\n    {msg}\n{'='*74}")
41

```

```

42
43 # -----
44 def stage_prep(force=False):
45     banner("STAGE: prep extracting mel-spectrograms")
46
47     us8k = load_us8k_metadata()
48     print(f"UrbanSound8K: {len(us8k)} clips, {us8k['class'].nunique()} classes"
49           )
50     X, meta = precompute_features(us8k, "us8k", force=force)
51
52     esc = load_esc50_metadata()
53     novel = esc[esc["category"].isin(C.ESC50_NOVEL_CLASSES)].reset_index(drop=
54         True)
55     print(f"ESC-50 novel subset: {len(novel)} clips, {novel['category'].nunique
56           ()} classes")
57     Xn, mn = precompute_features(novel, "esc50_novel", force=force)
58
59     _save("prep_summary", {
60         "us8k_shape": list(X.shape),
61         "us8k_class_counts": us8k["class"].value_counts().to_dict(),
62         "esc50_novel_shape": list(Xn.shape),
63         "esc50_novel_classes": sorted(novel["category"].unique().tolist()),
64         "feature": {"sample_rate": C.SAMPLE_RATE, "n_mels": C.N_MELS,
65                     "n_fft": C.N_FFT, "hop_length": C.HOP_LENGTH,
66                     "clip_seconds": C.CLIP_SECONDS, "n_frames": C.N_FRAMES},
67     })
68     return X, meta
69
70 # -----
71 def stage_explore(X, meta):
72     banner("STAGE: explore dataset figures")
73     viz.use_report_style()
74
75     us8k = load_us8k_metadata()
76     viz.plot_class_distribution(us8k["class"].value_counts().to_dict())
77
78     # one representative spectrogram per class
79     specs, titles = [], []
80     for cid in range(10):
81         rows = np.where(meta["classID"].values == cid)[0]
82         if len(rows):
83             specs.append(X[rows[0]])
84             titles.append(C.US8K_CLASSES[cid])
85     viz.plot_spectrogram_grid(specs, titles, ncols=5)
86
87     # the same class under augmentation, to illustrate what the model sees
88     from src.data import spec_augment, time_shift
89     rng = np.random.default_rng(C.SEED)
90     base = X[np.where(meta["classID"].values == 8)[0][0]] # siren
91     variants = [base, time_shift(base, rng), spec_augment(base, rng),
92                 spec_augment(time_shift(base, rng), rng)]
93     viz.plot_spectrogram_grid(
94         variants, ["original", "time shift", "SpecAugment", "both"],
95         name="fig_augmentation", ncols=4,
96         supitle="Augmentation: the same siren, varied so the CNN cannot
97             memorise it")
98
99 # -----
100 def stage_compare(X, meta, folds, epochs):
101     banner("STAGE: compare MLP vs CNN, and augmentation ablation")
102     from src.train import cross_validate, train_one_fold

```

```

101
102 results = {}
103 results["MLP"] = cross_validate(X, meta, "mlp", folds=folds, epochs=epochs,
104                                augment=False, tag="mlp")
105 results["CNN"] = cross_validate(X, meta, "cnn", folds=folds, epochs=epochs,
106                                augment=False, tag="cnn_noaug")
107 results["CNN + augment"] = cross_validate(X, meta, "cnn", folds=folds,
108                                           epochs=epochs,
109                                           augment=True, tag="cnn_aug")
110
111 viz.use_report_style()
112 viz.plot_model_comparison(results, n_folds=len(folds))
113
114 # training curves on a single representative fold
115 f = folds[0]
116 hists = {}
117 for label, (mname, aug) in {
118     "MLP": ("mlp", False),
119     "CNN": ("cnn", False),
120     "CNN + augment": ("cnn", True),
121 }.items():
122     _, res = train_one_fold(X, meta, test_fold=f, model_name=mname,
123                             epochs=epochs, augment=aug, verbose=False)
124     hists[label] = res.history
125 viz.plot_training_curves(hists)
126
127 _save("comparison_summary", results)
128 return results
129
130 # -----
131 def stage_final(X, meta, folds, epochs):
132     banner("STAGE: final full CNN evaluation")
133     from src.evaluate import evaluate_cv_predictions, full_metrics
134
135     y_true, y_pred, y_prob, per_fold = evaluate_cv_predictions(
136         X, meta, "cnn", folds=folds, epochs=epochs, augment=True)
137
138     m = full_metrics(y_true, y_pred, C.US8K_CLASSES)
139     print("\n" + m["report"])
140     print(f"Accuracy {m['accuracy']:.4f} | macro-F1 {m['f1_macro']:.4f}")
141
142     viz.use_report_style()
143     viz.plot_confusion_matrix(np.array(m["confusion_matrix"]), C.US8K_CLASSES)
144     viz.plot_per_class_f1(m["per_class_f1"])
145
146     _save("final_metrics", {k: v for k, v in m.items() if k != "report"})
147     (C.LOGS_DIR / "final_classification_report.txt").write_text(m["report"])
148
149     # Train and persist one deployable model (held-out fold 10) for the demo.
150     from src.train import train_one_fold
151     banner("training deployable model for the live demo (test fold 10)")
152     model, res = train_one_fold(X, meta, test_fold=10, model_name="cnn",
153                                 epochs=epochs, augment=True,
154                                 save_path=C.MODELS_DIR / "cnn_demo.pt")
155     print(f"demo model saved: test acc {res.test_acc:.4f}")
156     return m, model
157
158 # -----
159 def stage_cluster(X, meta, epochs):
160     banner("STAGE: cluster k-means + t-SNE on CNN embeddings")
161     from src.cluster import (cluster_purity, clustering_metrics, run_kmeans,

```

```

    run_tsne)
163 from src.evaluate import extract_embeddings
164 from src.train import train_one_fold
165
166 test_fold = 10
167 model, _ = train_one_fold(X, meta, test_fold=test_fold, model_name="cnn",
168                           epochs=epochs, augment=True, verbose=False)
169
170 (Xtr, ytr), _, (Xte, yte) = make_fold_split(X, meta, test_fold)
171 mean, std = compute_norm_stats(Xtr)
172
173 # Embeddings come from the held-out fold never data the CNN trained on.
174 emb, labels = extract_embeddings(model, Xte, yte, mean, std)
175 print(f"embeddings: {emb.shape}")
176
177 clusters, _ = run_kmeans(emb, n_clusters=10)
178 metrics = clustering_metrics(emb, clusters, labels)
179 purity = cluster_purity(clusters, labels, C.US8K_CLASSES)
180 print(f"ARI {metrics['adjusted_rand_index']:.3f} | "
181       f"NMI {metrics['normalized_mutual_info']:.3f} | "
182       f"silhouette {metrics['silhouette']:.3f}")
183
184 pts, idx = run_tsne(emb)
185 viz.use_report_style()
186 viz.plot_tsne(pts, labels[idx], C.US8K_CLASSES)
187
188 _save("cluster_metrics", {"metrics": metrics, "cluster_purity": purity})
189 return metrics
190
191
192 # -----
193 def stage_anomaly(X, meta, epochs):
194     banner("STAGE: anomaly autoencoder novelty detection")
195     from src.anomaly import (evaluate_novelty, reconstruction_scores,
196                             softmax_confidence_baseline, train_autoencoder)
197     from src.data import SpectrogramDataset
198     from src.train import predict, train_one_fold
199     from torch.utils.data import DataLoader
200
201     Xn = np.load(C.PROCESSED_DIR / "esc50_novel_X.npy")
202     print(f"novel (unseen) clips: {Xn.shape}")
203
204     test_fold = 10
205     (Xtr, ytr), (Xva, yva), (Xte, yte) = make_fold_split(X, meta, test_fold)
206     mean, std = compute_norm_stats(Xtr)
207
208     ae, hist = train_autoencoder(Xtr, mean, std, epochs=epochs, X_val=Xva)
209
210     known = reconstruction_scores(ae, Xte, mean, std)
211     novel = reconstruction_scores(ae, Xn, mean, std)
212     res = evaluate_novelty(known, novel)
213     print(f"AE novelty ROC-AUC {res['roc_auc']:.3f} | AP {res['
214           average_precision']:.3f} | "
215           f"P {res['precision']:.3f} R {res['recall']:.3f} F1 {res['f1']:.3f}")
216
217     # Baseline: does classifier under-confidence already do this job?
218     clf, _ = train_one_fold(X, meta, test_fold=test_fold, model_name="cnn",
219                             epochs=epochs, augment=True, verbose=False)
220     dl_k = DataLoader(SpectrogramDataset(Xte, yte, mean, std), batch_size=C.
221                       BATCH_SIZE)
222     dl_n = DataLoader(SpectrogramDataset(Xn, np.zeros(len(Xn), dtype=np.int64),
223                                           mean, std),
224                       batch_size=C.BATCH_SIZE)

```

```

222 _, _, pk = predict(clf, dl_k, C.DEVICE)
223 _, _, pn = predict(clf, dl_n, C.DEVICE)
224 base = softmax_confidence_baseline(pk, pn)
225 print(f"softmax baseline ROC-AUC {base['roc_auc']:.3f} "
226       f"(mean confidence: known {base['known_mean_conf']:.3f}, "
227       f"novel {base['novel_mean_conf']:.3f})")
228
229 viz.use_report_style()
230 viz.plot_anomaly_scores(known, novel, threshold=res["threshold"])
231 viz.plot_roc(np.array(res["fpr"]), np.array(res["tpr"]), res["roc_auc"])
232
233 # visual side-by-side of reconstructions
234 with torch.no_grad():
235     def recon(arr, i):
236         x = torch.from_numpy((arr[i] - mean) / std).float()[None, None].to(
237             C.DEVICE)
238         return x[0, 0].cpu().numpy(), ae(x)[0, 0].cpu().numpy()
239
240     pairs = [recon(Xte, 0), recon(Xte, 1), recon(Xn, 0), recon(Xn, 1)]
241     viz.plot_reconstructions([p[0] for p in pairs], [p[1] for p in pairs],
242                             ["known", "known", "novel", "novel"])
243
244 torch.save({"state_dict": ae.state_dict(), "mean": mean, "std": std,
245            "threshold": res["threshold"]}, C.MODELS_DIR / "autoencoder.pt"
246           )
247
248 _save("anomaly_results", {
249     "autoencoder": {k: v for k, v in res.items() if k not in ("fpr", "tpr")
250 },
251     "softmax_baseline": base,
252     "ae_history": hist,
253 })
254 return res
255
256 # -----
257 def main():
258     ap = argparse.ArgumentParser(description=__doc__,
259                                formatter_class=argparse.
260                                    RawDescriptionHelpFormatter)
261     ap.add_argument("--stage", default="all",
262                    choices=["all", "prep", "explore", "compare", "final",
263                             "cluster", "anomaly"])
264     ap.add_argument("--folds", type=int, nargs="+", default=None,
265                    help="folds to evaluate (default: all 10)")
266     ap.add_argument("--epochs", type=int, default=C.EPOCHS)
267     ap.add_argument("--quick", action="store_true",
268                    help="fast smoke run: 1 fold, few epochs")
269     ap.add_argument("--force-prep", action="store_true")
270     args = ap.parse_args()
271
272     folds = args.folds or list(range(1, C.N_FOLDS + 1))
273     epochs = args.epochs
274     if args.quick:
275         folds, epochs = [1], 3
276         print(">>> QUICK MODE: 1 fold, 3 epochs")
277
278     print(f"device: {C.DEVICE} | torch {torch.__version__}")
279     print(f"folds: {folds} | epochs: {epochs}")
280     t0 = time.time()
281
282     X, meta = stage_prep(force=args.force_prep)
283     s = args.stage

```

```

281
282     if s in ("all", "explore"):
283         stage_explore(X, meta)
284     if s in ("all", "compare"):
285         stage_compare(X, meta, folds, epochs)
286     if s in ("all", "final"):
287         stage_final(X, meta, folds, epochs)
288     if s in ("all", "cluster"):
289         stage_cluster(X, meta, epochs)
290     if s in ("all", "anomaly"):
291         stage_anomaly(X, meta, epochs)
292
293     banner(f"DONE in {(time.time()-t0)/60:.1f} min    figures in {C.FIGURES_DIR}"
294            )
295
296 if __name__ == "__main__":
297     main()

```

Listing 14: Staged experiment runner (run_experiments.py)

D.10 demo/live_demo.py

Real-time microphone demo.

```

1  #!/usr/bin/env python
2  """Real-time acoustic scene awareness from the laptop microphone.
3
4  Holds a rolling 4-second window of audio, converts it to a log-mel spectrogram
5  on the same front-end used in training, and runs both networks each refresh:
6
7  * the CNN classifies the sound and drives the confidence bars
8  * the autoencoder scores how familiar it is, so an unrecognised sound is
9  reported as UNKNOWN rather than forced into one of the ten labels
10
11  Run:
12      python demo/live_demo.py                # live microphone
13      python demo/live_demo.py --list-devices
14      python demo/live_demo.py --file clip.wav # replay a file instead
15  """
16  from __future__ import annotations
17
18  import argparse
19  import queue
20  import sys
21  import threading
22  from pathlib import Path
23
24  import matplotlib
25  import matplotlib.pyplot as plt
26  import numpy as np
27  import torch
28
29  sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
30
31  from src import config as C  # noqa: E402
32  from src import viz  # noqa: E402
33  from src.data import audio_to_melspec  # noqa: E402
34  from src.models import AudioCNN, ConvAutoencoder  # noqa: E402
35
36  REFRESH_SEC = 0.5  # how often inference runs
37  SMOOTHING = 0.6  # EMA over probabilities, damps single-frame flicker
38

```

```

39
40 # -----
41 class Engine:
42     """Loads the trained models and turns a waveform into a prediction."""
43
44     def __init__(self, cnn_path, ae_path=None, device=C.DEVICE):
45         self.device = device
46
47         ck = torch.load(cnn_path, map_location=device, weights_only=False)
48         self.mean, self.std = ck["mean"], ck["std"]
49         self.cnn = AudioCNN(n_classes=ck.get("n_classes", 10)).to(device)
50         self.cnn.load_state_dict(ck["state_dict"])
51         self.cnn.eval()
52
53         self.ae = None
54         self.threshold = None
55         if ae_path and Path(ae_path).exists():
56             ack = torch.load(ae_path, map_location=device, weights_only=False)
57             self.ae = ConvAutoencoder().to(device)
58             self.ae.load_state_dict(ack["state_dict"])
59             self.ae.eval()
60             self.threshold = ack.get("threshold")
61
62         self.probs = np.zeros(len(C.US8K_CLASSES))
63
64     @torch.no_grad()
65     def infer(self, wave: np.ndarray):
66         spec = audio_to_melspec(wave)
67         x = torch.from_numpy((spec - self.mean) / self.std).float()[None, None]
68             .to(self.device)
69
70         p = torch.softmax(self.cnn(x), dim=1)[0].cpu().numpy()
71         self.probs = SMOOTHING * self.probs + (1 - SMOOTHING) * p # EMA
72
73         novelty, is_novel = None, False
74         if self.ae is not None:
75             novelty = float(self.ae.reconstruction_error(x).item())
76             if self.threshold is not None:
77                 is_novel = novelty > self.threshold
78
79         return spec, self.probs, novelty, is_novel
80
81 # -----
82 class AudioStream:
83     """Rolling microphone buffer of the last CLIP_SECONDS seconds."""
84
85     def __init__(self, device=None, samplerate=C.SAMPLE_RATE):
86         self.samplerate = samplerate
87         self.buffer = np.zeros(C.N_SAMPLES, dtype=np.float32)
88         self.lock = threading.Lock()
89         self.q: queue.Queue = queue.Queue()
90         self.device = device
91         self.stream = None
92         self.level = 0.0
93
94     def _callback(self, indata, frames, time_info, status):
95         if status:
96             print(f"[audio] {status}", file=sys.stderr)
97         mono = indata[:, 0].copy()
98         with self.lock:
99             n = len(mono)
100             self.buffer = np.roll(self.buffer, -n)

```

```

101         self.buffer[-n:] = mono
102         self.level = float(np.abs(mono).max())
103
104     def start(self):
105         import sounddevice as sd
106
107         self.stream = sd.InputStream(
108             channels=1, samplerate=self.samplerate, device=self.device,
109             blocksize=int(self.samplerate * 0.1), callback=self._callback,
110         )
111         self.stream.start()
112         print(f"[audio] listening at {self.samplerate} Hz")
113
114     def read(self) -> np.ndarray:
115         with self.lock:
116             return self.buffer.copy()
117
118     def stop(self):
119         if self.stream:
120             self.stream.stop(); self.stream.close()
121
122
123 # -----
124 class Dashboard:
125     """Live figure: spectrogram, confidence bars, and a status banner."""
126
127     def __init__(self, engine: Engine, source, is_file=False):
128         self.engine = engine
129         self.source = source
130         self.is_file = is_file
131         viz.use_report_style()
132
133         self.fig = plt.figure(figsize=(13, 6.6))
134         self.fig.canvas.manager.set_window_title("Acoustic Scene Awareness
135             live")
136         gs = self.fig.add_gridspec(2, 2, height_ratios=[1, 5.2],
137             width_ratios=[1.35, 1], hspace=0.28, wspace
138                 =0.22)
139
140         # Status banner
141         self.ax_banner = self.fig.add_subplot(gs[0, :])
142         self.ax_banner.axis("off")
143         self.banner = self.ax_banner.text(
144             0.5, 0.5, "listening", ha="center", va="center",
145             fontsize=25, fontweight="bold", color=viz.INK,
146             transform=self.ax_banner.transAxes,
147             bbox=dict(boxstyle="round,pad=0.55", fc="#eef3fa", ec=viz.SERIES
148                 [0], lw=2.5))
149
150         # Spectrogram
151         self.ax_spec = self.fig.add_subplot(gs[1, 0])
152         self.im = self.ax_spec.imshow(
153             np.full((C.N_MELS, C.N_FRAMES), -80.0), origin="lower", aspect="
154                 auto",
155             cmap="magma", vmin=-80, vmax=0, extent=[0, C.CLIP_SECONDS, 0, C.
156                 N_MELS])
157         self.ax_spec.set_title("Live log-mel spectrogram (last 4 s)", loc="left
158             ", pad=10)
159         self.ax_spec.set_xlabel("Time (s)"); self.ax_spec.set_ylabel("Mel band"
160             )
161         self.ax_spec.grid(False)
162
163         # Confidence bars

```



```

157     self.axBars = self.fig.add_subplot(gs[1, 1])
158     labels = [C.DEMO_LABELS[c] for c in C.US8K_CLASSES]
159     self.bars = self.axBars.barh(labels, np.zeros(len(labels)),
160                                color=viz.SERIES[0], height=0.62)
161     self.val_txt = [
162         self.axBars.text(0.6, b.get_y() + b.get_height() / 2, "", va="
163             center",
164                        fontsize=9, color=viz.INK_2)
165         for b in self.bars
166     ]
167     self.axBars.set_xlim(0, 1)
168     self.axBars.set_xlabel("Confidence")
169     self.axBars.set_title("CNN class confidence", loc="left", pad=10)
170     self.axBars.grid(axis="y", visible=False)
171     for s in ("top", "right"):
172         self.axBars.spines[s].set_visible(False)
173
174     self.file_pos = 0
175
176     def _next_wave(self) -> np.ndarray:
177         if not self.is_file:
178             return self.source.read()
179         # File mode: sweep a window across the clip so the demo animates.
180         wave = self.source
181         step = int(C.SAMPLE_RATE * REFRESH_SEC)
182         start = self.file_pos % max(len(wave), 1)
183         self.file_pos += step
184         w = wave[start:start + C.N_SAMPLES]
185         if len(w) < C.N_SAMPLES:
186             w = np.pad(w, (0, C.N_SAMPLES - len(w)))
187         return w
188
189     def update(self, _frame):
190         wave = self._next_wave()
191
192         if np.abs(wave).max() < 1e-4: # effectively silence
193             self.banner.set_text("listening")
194             self.banner.set_color(viz.INK)
195             self.banner.get_bbox_patch().set(fc="#f2f2ef", ec=viz.AXIS)
196             for b, t in zip(self.bars, self.val_txt):
197                 b.set_width(0); t.set_text("")
198             return
199
200         spec, probs, novelty, is_novel = self.engine.infer(wave)
201         self.im.set_data(spec)
202
203         top = int(np.argmax(probs))
204         conf = float(probs[top])
205         cls = C.US8K_CLASSES[top]
206
207         for b, t, p in zip(self.bars, self.val_txt, probs):
208             b.set_width(p)
209             b.set_color(viz.SERIES[0] if p < 0.5 else viz.SERIES[1])
210             t.set_x(min(p + 0.02, 0.88))
211             t.set_text(f"{p*100:.0f}%" if p > 0.02 else "")
212
213         if is_novel:
214             txt, fc, ec, col = (" UNKNOWN SOUND", "#fdf3e3",
215                                viz.STATUS["warning"], "#7a5200")
216         elif cls in C.ALERT_CLASSES and conf > 0.45:
217             txt = f" {C.DEMO_LABELS[cls].upper()}"
218             fc, ec, col = "#fdeaea", viz.STATUS["critical"], "#8c1d1d"
219         elif conf > 0.35:

```

```

219         txt, fc, ec, col = (C.DEMO_LABELS[cls], "#eef3fa", viz.SERIES[0],
220                             viz.INK)
221     else:
222         txt, fc, ec, col = ("listening", "#f2f2ef", viz.AXIS, viz.INK)
223
224     if conf > 0.35 and not is_novel:
225         txt += f" {conf*100:.0f}%"
226     if novelty is not None:
227         txt += f" novelty {novelty:.3f}"
228
229     self.banner.set_text(txt)
230     self.banner.set_color(col)
231     self.banner.get_bbox_patch().set(fc=fc, ec=ec)
232
233 def run(self):
234     from matplotlib.animation import FuncAnimation
235
236     self.anim = FuncAnimation(self.fig, self.update,
237                               interval=int(REFRESH_SEC * 1000),
238                               cache_frame_data=False)
239
240     plt.tight_layout()
241     plt.show()
242
243 # -----
244 def main():
245     ap = argparse.ArgumentParser(description=__doc__,
246                                 formatter_class=argparse.
247                                     RawDescriptionHelpFormatter)
248
249     ap.add_argument("--model", default=str(C.MODELS_DIR / "cnn_demo.pt"))
250     ap.add_argument("--autoencoder", default=str(C.MODELS_DIR / "autoencoder.pt"))
251
252     ap.add_argument("--device", type=int, default=None, help="input device index")
253     ap.add_argument("--file", default=None, help="replay a wav file instead of the mic")
254     ap.add_argument("--list-devices", action="store_true")
255     args = ap.parse_args()
256
257     if args.list_devices:
258         import sounddevice as sd
259         print(sd.query_devices())
260         return
261
262     if not Path(args.model).exists():
263         sys.exit(f"Model not found: {args.model}\n"
264                 f"Train one first: python run_experiments.py --stage final")
265
266     # pyplot is already imported (via src.viz), so the backend has to be
267     # switched rather than set matplotlib.use() would be a no-op here.
268     for backend in (("MacOSX", "TkAgg") if sys.platform == "darwin" else ("
269         TkAgg",)):
270         try:
271             plt.switch_backend(backend)
272             break
273         except Exception:
274             continue
275
276     if matplotlib.get_backend().lower() in ("agg", "template"):
277         sys.exit("No interactive matplotlib backend available the live demo "
278                 "needs a display. Try: uv pip install pyqt6")
279     print(f"[ui] backend: {matplotlib.get_backend()}")
280
281     engine = Engine(args.model, args.autoencoder)

```

```
276     print(f"[model] loaded on {engine.device}")
277         f"{' (+ autoencoder)' if engine.ae is not None else ''}")
278
279     if args.file:
280         import librosa
281         wave, _ = librosa.load(args.file, sr=C.SAMPLE_RATE, mono=True)
282         print(f"[file] {args.file} {len(wave)/C.SAMPLE_RATE:.1f}s")
283         Dashboard(engine, wave, is_file=True).run()
284         return
285
286     stream = AudioStream(device=args.device)
287     stream.start()
288     try:
289         Dashboard(engine, stream).run()
290     finally:
291         stream.stop()
292         print("[audio] stopped")
293
294
295 if __name__ == "__main__":
296     main()
```

Listing 15: Real-time microphone demo (demo/live_demo.py)